

Final Thesis Report

Anchor-Guided Repair: A Defense Mechanism for Enhancing Stability of Compromised Pretrained Language Models Against Low-Precision and Weight Noise Attacks

by

Abrar Mahir Rohan

22101267

Anika Bushra

21201068

Fuad Fardin

22101027

Nafiz Khan

22101045

Tahsin Tajwar Tanni

22101744

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
January 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



Abrar Mahir Rohan

22101267



Anika Bushra

21201068



Fuad Fardin

22101027



Nafiz Khan

22101045



Tahsin Tajwar Tanni

22101744

Approval

The thesis/project titled “Anchor-Guided Repair: A Defense Mechanism for Enhancing Stability of Compromised Pretrained Language Models Against Low-Precision and Weight Noise Attacks” submitted by

1. Abrar Mahir Rohan (22101267)
2. Anika Bushra (21201068)
3. Fuad Fardin (22101027)
4. Nafiz Khan (22101045)
5. Tahsin Tajwar Tanni (22101744)

of Fall 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in 31st January, 2026.

Examining Committee:

Supervisor:
(Member)



Dr. Muhammad Iqbal Hossain

Associate Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

Large Language Models (LLMs) are increasingly released as open-source and are susceptible to post-release attacks, including weight noise injection and low-precision quantization. Such attacks often have a significant negative effect on model stability and performance, augmenting perplexity and creating unreliable behavior. In practice, users might not have access to original clean weights and may unknowingly download compromised models. This thesis presents a defense mechanism called Anchor-Guided Repair, which aims to improve the stability of weakened LLMs against weight noise and low-precision attacks. The proposed method optimizes a fine-tuned attacked model on clean textual data while mitigating parameter changes via an anchor loss, which penalizes the difference from a clean, task-adapted baseline. This approach reduces instability by optimizing a composite task involving language modeling loss and anchor regularization, without distorting the knowledge acquired by the original model. The proposed method was put to the test across a range of model architectures and attack setups, including low-precision and Gaussian weight noise. The experimental results clearly show that Anchor-Guided Repair outperforms the attacked models consistently, keeping a favorable state of desirable conditions. While the defended model performs slightly below the clean baseline, it is significantly better than the compromised model. This emphasizes the power of anchoring in stabilizing large models without accessing proprietary training data. Altogether, Anchor-Guided Repair is a viable post-deployment defense for weight-level attacks, promoting safer and more trustworthy model reuse in open-source settings.

Keywords: Quantization attacks; Large Language Models security; weight noise injection; adversarial robustness; model compression; backdoor detection.

Acknowledgement

It is our great pleasure to extend our sincere appreciation to our advisor Dr. Muhammad Iqbal Hossain who gave us the most excellent guidance and patience during the course of this thesis and offered us excellent comments as well. His support and experience have proved to be priceless through all the steps of this work. The present study has been conducted with the material available at the University Library to which we are thankful.

Personally, we would like to express our appreciation to our parents, siblings who stood behind, showed love and support, and patience throughout the process. It is not possible without their support.

Thank you all.

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	viii
Nomenclature	ix
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study	1
1.3 Problem Statement	2
1.4 Objectives	3
1.4.1 General Objective:	3
1.4.2 Specific Objectives:	3
1.5 Methodology in Brief	3
1.6 Scopes and Challenges	5
1.6.1 What This Study Covers:	5
1.6.2 What This Study Does not covers:	5
1.6.3 Key Challenges:	6
2 Literature Review	7
2.1 Literature Review on Low-Precision and Weight Noise Attacks in LLMs	7
2.1.1 Introduction to Low-Precision Computation and Gaussian Noise Attacks	7
2.1.2 Review of Existing Research	7
2.2 Summary of Key Findings	17
2.2.1 Quantization as a Security Vulnerability	17
2.2.2 Weight Sensitivity and the “Anchor” Hypothesis	18
2.2.3 Robustness Against Weight Noise and Perturbations	18
2.2.4 Conclusion and Correlation with Thesis Objective	18

3	Requirements, Impacts and Constraints	19
3.1	Final Specifications and Requirements	19
3.2	Societal Impact	21
3.3	Environmental Impact	22
3.4	Ethical Issues	22
3.5	Standards	22
3.6	Project Management Plan	23
3.7	Risk Management	23
3.8	Economic Analysis	23
4	Proposed Methodology	25
4.1	Design Process or Methodology Overview	25
4.1.1	Baseline Establishment and Initial Performance Assessment	25
4.1.2	Task Adaptation and Clean Baseline Establishment	27
4.1.3	Attack Simulation Framework	28
4.1.4	Defense Training Through Anchored Fine-Tuning	30
4.1.5	Multi-Precision Evaluation and Performance Analysis	31
4.2	Preliminary Design or Design (Model) Specification	32
4.2.1	Theoretical Framework: Quantization and Perplexity	32
4.2.2	Data Sources and Evaluation Inputs	34
4.2.3	Implementation Details of Selected Design	35
5	Result Analysis	39
5.1	Performance Evaluation	39
5.1.1	Evaluation Setup	39
5.1.2	Results	39
5.2	Analysis of Design Solutions	44
5.3	Final Design Adjustment	46
5.4	Statistical Analysis	46
5.4.1	Prompt-wise Ordering Consistency	47
5.4.2	Stability / Variance Analysis	49
5.5	Comparisons and Relationships	52
5.5.1	Qwen vs Pythia	52
5.5.2	Relationship Between Noise Level and Recovery	52
5.5.3	Relationship Between Precision and Recovery	52
5.6	Discussion	52
5.6.1	Why Task-Adapted Baseline Methodology Produces Interpretable Recovery	52
5.6.2	Comparative Semantic Analysis (t-SNE)	53
5.6.3	Comparative Semantic Analysis (t-SNE)	53
5.6.4	Comparison with Existing Defense Mechanisms	54
6	Conclusion	56
6.1	Summary of Findings	56
6.2	Contributions to the Field	57
6.3	Recommendations for Future Work	57
	Bibliography	60

List of Figures

4.1	Experimental Methodology Flowchart.	26
4.2	Impact of Gaussian Weight Noise on Perplexity Degradation across Varying Noise Intensities (10%, 15%, and 20%) for Qwen2.5-0.5B. . .	29
4.3	Impact of Gaussian Weight Noise on Perplexity Degradation across Varying Noise Intensities (10%, 15%, and 20%) for Pythia-410M. . . .	30
4.4	Operational Workflow of the Anchor-Guided Repair Mechanism. . . .	37
5.1	1st attempt results for Qwen2.5-0.5B(20% noise)	40
5.2	2nd attempt results for Qwen2.5-0.5B(20% noise)	41
5.3	3rd attempt results for Qwen2.5-0.5B(20% noise)	41
5.4	1st attempt results for Pythia-410M(20% noise)	43
5.5	2nd attempt results for Pythia-410M(20% noise)	43
5.6	3rd attempt results for Pythia-410M(20% noise)	45
5.7	Prompt-wise Ordering Sanity Check for Qwen2.5-0.5B.	48
5.8	Prompt-wise Ordering Sanity Check for Pythia-410M.	49
5.9	Variance Sanity Check for Qwen2.5-0.5B.	50
5.10	Variance Sanity Check for Pythia-410M.	51
5.11	Comparative t-SNE Visualization at 20% Noise Intensity.	54

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

ADC Analog-to-Digital Converter

AWQ Activation-aware Weight Quantization

CFE Counterfactual Example

IMC In-Memory Computing

LoRA Low-Rank Adaptation

MMLU Massive Multitask Language Understanding

NIT Noise Injection-based Training

NLE Natural Language Explanation

NPFT Noise Perturbation Fine-tuning

OSP Outlier-Safe Pre-Training

OSTQuant Orthogonal and Scaling Transformations Quantization

PEFT Parameter-Efficient Fine-Tuning

PGD Projected Gradient Descent

PTQ Post-Training Quantization

QAT Quantization-Aware Training

QSUR Quantization Space Utilization Rate

SLIM Sparsity and Low-rank approximation for LLM weight compression

SpQR Sparse-Quantized Representation

Chapter 1

Introduction

1.1 Background

The rapid proliferation of Large Language Models (LLMs) has fundamentally transformed the landscape of natural language processing, enabling widespread adoption across domains ranging from code generation to creative writing. However, the immense computational and memory costs associated with modern LLMs often exceeding billions of parameters pose significant barriers to deployment on commodity hardware [6]. To address this, the open-source ecosystem has increasingly relied on Post-Training Quantization (PTQ) techniques, such as LLM.int8() and NF4, which compress model weights from standard floating-point formats (FP16/FP32) into lower-precision representations (INT8 or 4-bit) [2], [17]. This shift has democratized access to powerful AI, yet it introduces new challenges regarding the handling of activation outliers and weight precision loss [7], [18].

While quantization plays a crucial role in enhancing efficiency, it also opens up a significant vulnerability: the sensitivity of model weights. Recent research has shown that popular quantization techniques can be manipulated, leading to harmful outcomes. Even minor tweaks to weight parameters can cause a drastic drop in model performance or security [5]. As models are compressed into lower bitwidths, the “safety margin” for these weights shrinks, making them more prone to instability caused by noise [15]. As a result, the integrity of the open-source supply chain heavily depends on the belief that downloaded weights are untouched and reliable. However, this assumption is increasingly being put to the test with the rise of quantization backdoors and weight-level attacks [9], [13].

1.2 Rationale of the Study

The primary motivation for this research stems from a critical gap in the current defense landscape for open-source LLMs: the lack of *user-side* repair mechanisms. Existing robustness techniques, such as noise injection-based training schemes or aggressive quantization-aware training, are predominantly *creator-side* defenses [1], [4]. These methods require access to the full proprietary training dataset and massive computational resources to train a robust model from scratch, which is infeasible for the average end-user.

However, in the real-world open-source ecosystem, users typically download pre-trained models that may already be compromised by weight noise or structural instability. When a user encounters a degraded model (e.g., one suffering from high perplexity due to quantization noise), they lack the resources to retrain it and often lack access to the original “clean” weights for comparison [14]. This creates a dilemma where users must either discard the model or deploy unreliable systems that may exhibit degraded code quality or reasoning capabilities [11].

Therefore, there is an urgent need for a post-deployment defense mechanism that allows end-users to “repair” compromised models using limited data and compute. By developing a method that can stabilize weights against noise without requiring access to the original training pipeline, this study aims to secure the deployment of quantized LLMs [20].

1.3 Problem Statement

Large language models (LLMs) are distributed more and more often in open-source repositories and third-party platforms, which allows their wide adoption, but also creates additional vulnerabilities in the model supply chain. In reality, models often get deployed by end users that have been subjected to aggressive post-training changes like low-precision quantization or unintentional corruption of the parameters in the course of storage, transmission or conversion. More importantly, these models could be purposefully weakened by intentional weight fluctuations, which could lead to inadequate performance, unstable generation, or silent failure modes that are challenging to identify during deployment.

Conventional recovery techniques like full retraining or reference-based rectification are impractical because end users usually do not have accessibility to the basic clean pretrained weights or proprietary training data. Unconstrained optimisation frequently results in catastrophic parameter drift, overfitting, or unexpected behavioural changes, particularly when the model has already been damaged by quantisation or weight noise. However, fine-tuning on downstream data may partially restore performance.

Current robustness and repair methods mostly presuppose oracle access to clean model parameters or entirely rely on performance measures aggregated on in-distribution data. The actual limitations of post-deployment repair have been obscured by these assumptions, which also fail to account for task generalisation and the delicate negotiations between computational stability and representational preservation. In addition, the effectiveness of recovery in mixed-precision regimes especially with extreme low-bit quantization has not been studied well yet, even as it has increasingly become an important phenomenon in resource-constrained deployment environments.

As such, whether or not compromised pretrained language models can be stabilized and partially recovered at deployment time under realistic user-side constraints, without conditioning on clean reference weights or re-training on a clean dataset, and coherent behaviour on both in-distribution and out-of-distribution tasks is of critical open interest. To deal with this issue, a principled repair process that limits

the damaging parameter updates, as well as a stringent evaluation framework that can be used to identify the superficiality of the performance improvement and the true recovery of the behaviour, is necessary.

1.4 Objectives

1.4.1 General Objective:

The main aim of the study is to research the possibility of reliably stabilizing and partially recovering at deployment time compromised pretrained large language models without the availability of proprietary training data, or original clean weights, under plausible post-release threat conditions.

1.4.2 Specific Objectives:

In order to accomplish this broad objective, the study will aim at the following specific objectives:

1. To provide an anchor-guided repair model that allows previously trained language models to recover from low-precision quantisation errors and Gaussian weight noise while simultaneously enhancing language models using parameter-level stability restrictions.
2. To formulate and study a two objective loss that trades off between task performance recovery and preserving representations so that parameter drift can be disastrous upon fine-tuning after the attack.
3. To empirically test a realistic user-side defensive strategy versus oracle-based upper-bound defensive, have the ability to measure the recovery that can be achieved when no clean reference weights are available and have meaningful bounds on the post-deployment repair.
4. In order to comprehensively assess robustness recovery in a variety of numerical precision regimes (FP16, INT8 and 4-bit), one would evaluate the amplification or limiting effects of quantization on recovery dynamics with increasing noise severity.
5. Based on data-level perplexity, prompt-wise ordering consistency, variance optimisation, and semantic embedding analysis, both in-distribution and out-of-distribution stability are assessed to determine whether recovered models continue to behave coherently outside the fine-tuning domain.
6. To evaluate instruction-tuned and basic pretrained models in order to examine the impact of model design and pretraining alignment on repair performance.

1.5 Methodology in Brief

The methodology adopted in this study follows a structured evaluation pipeline designed to analyze robustness degradation and recovery in pretrained language models

under weight-noise perturbations. The process is divided into four main stages, consistent with the proposed experimental flowchart.

In the baseline establishment stage, a pretrained language model (Qwen2.5-0.5B-Instruct or Pythia-410M) undergoes task-specific adaptation through fine-tuning on the WikiText-2 dataset to establish a task-adapted clean baseline.

Language modeling performance is gauged using perplexity on the official test split of WikiText-2. This phase sets a solid performance benchmark by ensuring that the clean reference model is tailored to the evaluation task. This way, we avoid those nonsensical recovery rates that can go over 100% when the defended models are fine-tuned on the same dataset.

In the attack generation stage, the task-adapted baseline model is subjected to controlled weight corruption through Gaussian noise injection. In order to approximate the adversarial perturbations, Gaussian noise with zero mean and variance σ^2 is introduced to adapted model weights which is shown in Equation 1.1. This method allows known testing of the robustness of the model in the case of stochastic disturbance.

$$W_{\text{attacked}} = W_{\text{clean.adapted}} + \mathcal{N}(0, \sigma^2) \quad (1.1)$$

The noise variance, σ^2 , is adjusted to hit target performance degradation levels of 10%, 15%, and 20% increase in perplexity compared to the clean adapted baseline. The attacked model is saved in half precision (FP16) and later evaluated under post-training quantization settings (INT8 and 4-BIT) to measure performance loss and understand how weight noise and quantization errors compound.

The defense and recovery stage applies a noise-aware fine-tuning strategy using a regularized objective that combines language modeling loss with a weight anchoring constraint. The overall optimization problem is made of the standard cross-entropy loss and a regularization term that discourages deviation on the clean adapted weights. Hyperparameter λ is used to balance between trade-off between task performance and weight stability that we can see in Equation 1.2.

$$L_{\text{total}} = L_{CE} + \lambda \times \sum \text{mean}((W_{\text{current}} - W_{\text{clean.adapted}})^2) \quad (1.2)$$

This method allows for performance recovery while keeping excessive parameter drift from the task-adapted clean baseline in check. The clean adapted model acts as a stable reference point (stored on CPU for memory efficiency), while the attacked model remains trainable (on GPU). Fine-tuning is carried out on WikiText-2 with hyperparameters set to $\lambda = 0$, a learning_rate = $2e-5$, and a maximum of 200 steps. Parameter updates are done in half precision (FP16), with post-training quantization applied only during the final evaluation.

Finally, the evaluation stage extends beyond dataset-level metrics through three complementary analyses. Dataset-level perplexity is measured at FP16, INT8, and

4-BIT precisions, with recovery rate calculated as:

$$\text{Recovery} = \frac{PPL_{\text{attacked}} - PPL_{\text{defended}}}{PPL_{\text{attacked}} - PPL_{\text{clean.adapted}}} \times 100\% \quad (1.3)$$

The relative improvement of the defense mechanism over the attacked model is the definition of recovery in Equation 1.3 and it is calculated by the performance difference between the attacked and clean adapted models. The greater the recovery percentage, the better the mitigating efforts to address the attack.

A suite of 96 prompts across various task categories facilitates prompt-wise perplexity analysis, validating ordering constraints ($PPL_{\text{clean}} < PPL_{\text{defended}} < PPL_{\text{attacked}}$) and measuring variance reduction to evaluate stability improvements. We also look at semantic consistency through embedding extraction and t-SNE dimensionality reduction, which helps visualize the geometric relationships between clean, attacked, and defended model representations, confirming that the defense restores semantic structure. These analyses affirm the effectiveness of the defense mechanism across both Qwen2.5-0.5B-Instruct and Pythia-410M architectures.

1.6 Scopes and Challenges

1.6.1 What This Study Covers:

The study aims at understanding the degradation and recovery of robustness in pretrained language models to the gaussian noise and low precision attack. It also explores performance, stability and how much the meaning is held to with controlled disturbances and all with standardized datasets and structured prompt assessments. This paper highlights the importance of empirical analysis rather than redesigning architectures or retraining at an extensive basis.

Technical Focus

- The effect of low precision and noise attack on the loss of robustness.
- Controlled recovering through regularized fine-tuning
- Timely-wise performance and variance analysis.
- Maintaining semantic embeddings.
- Benchmarking on different model families.

1.6.2 What This Study Does not covers:

- The attacker can attack prompt injection or data poisoning.
- Changes to transformer model structures
- Multilingual evaluation settings
- Commercial deployment pipelines or system-level defenses

1.6.3 Key Challenges:

- **Trade Off: Recovery vs Stability** It is hard to recover performance after quantizing without introducing excessive drift in the parameters. By over-adapting we may get unstable or even overfitted, whereas by under-adapting we may fail to make much progress.
- **Computational Cost at Prompt-Level:** Fine-grained prompt-wise analysis may indeed cause a significant increase in computational expenses, particularly when measuring the sensitivity to variability in levels of noise and variability in models.
- **Interpreting Embedding Visualizations:** Techniques for reducing dimensions can preserve local structures but can distort global distances so we must carefully interpret them not to overstate semantic similarities.

In spite of these challenges, the given study provides a firm and comprehensive layout of determining the extent to which pretrained language models can restore their resilience.

Chapter 2

Literature Review

2.1 Literature Review on Low-Precision and Weight Noise Attacks in LLMs

In this part, the literature examines the current studies on low-precision computation and weight noise perturbation in Large Language Models (LLMs) and their effects on model stability, robustness, and security. Reducing numerical precision (i.e. FP16 to INT8 or INT4) and adding noise, both accidentally during hardware limitations and deliberately during adversarial attacks, have been found by prior work to cause the performance and reliability of a model to decline significantly. This knowledge of their effects is essential to develop an efficient defense mechanism, e.g. anchor-guided repair, against the compromised models.

2.1.1 Introduction to Low-Precision Computation and Gaussian Noise Attacks

Low-precision computation is also common in deep learning models to achieve smaller memory footprint and computation by quantizing model parameters and activations with smaller bit-widths FP16, INT8 or INT4. Though these methods allow an effective implementation of LLMs on resource-limited hardware, they may cause numerical instability and accuracy reduction.

Besides the loss of precision, the presence of Gaussian noise in model weights, whether as a result of hardware bugs, transmission noise, or as a result of adversarial attacks, is also a significant vulnerability to model robustness. Weight noise attacks are based on the fact that the parameters of LLM are sensitive to noise, and the resulting performance collapses with noise or the outputs become unpredictable. This section gives background information regarding low-precision arithmetic as well as Gaussian noise, which is critical to the behavior of LLM, which paves the way to defense-oriented research.

2.1.2 Review of Existing Research

Patrik Czako et al. [18] conducted a systematic review focusing on weight-activation quantization in Large Language Models (LLMs), with a specific emphasis on the emergent outlier phenomenon in LLM activations. Their primary purpose was to

evaluate and synthesize recent techniques designed to mitigate these activation outliers and enhance quantization efficiency, distinguishing their work from previous reviews by this unique focus. To achieve this, they meticulously reviewed 52 recent studies following the PRISMA methodology. This involved a structured literature search across academic databases using specific keywords and strict eligibility criteria, ensuring selection of papers proposing practical post-training quantization (PTQ) methods for multi-billion parameter LLMs that address activation and KV cache challenges. Their main contribution is the presentation of a diverse set of techniques and their implications for activation quantization, laying foundational groundwork for future research and practical advancements in LLM deployment. They categorized these techniques into mathematically equivalent transformations (such as channel-wise scaling, shifting, and rotation), high-precision storage (mixed precision), finer granularity (smaller quantization groups), and non-linear datatypes. The review’s outcome revealed that while mathematically equivalent transformations, especially channel-wise scaling, partially mitigate outliers by shifting quantization difficulty to weights, techniques like channel-wise shifting and rotation address more complex outliers (e.g., massive outliers). Mixed-precision techniques were shown to be crucial for KV cache quantization, with methods like GEAR and ZipCache performing well. Despite advancements, the analysis indicated that weight-only quantization still outperforms weight-activation techniques in minimizing accuracy loss, but activation quantization remains vital for optimal inference speed and energy efficiency. The studies reviewed frequently utilized datasets such as WikiText-2 for perplexity evaluations and GSM8k for accuracy assessments, both of which are publicly available.

Kazuki Egashira et al. [5] investigated the adverse security implications of widely used LLM quantization methods, demonstrating that these techniques can be exploited to create a harmful quantized LLM even if the full-precision version appears benign. Their core purpose was to reveal this novel threat, which could potentially trick users into deploying malicious quantized models. They addressed this problem through a three-stage attack framework. First, they injected malicious behavior into an LLM via fine-tuning on an adversarial task (e.g., vulnerable code generation). Second, they quantized this malicious model to calculate constraints defining all full-precision models that would quantize to the same malicious state. They focused on popular zero-shot methods like LLM.int8(), NF4, and FP4, which normalize weights within blocks and round them to a defined alphabet, allowing derivation of these critical constraints. Third, they used Projected Gradient Descent (PGD) to tune out the malicious behavior from the full-precision model while ensuring its weights remained within the derived constraints, resulting in a benign full-precision model that, upon quantization, would revert to its malicious form. Their main contribution is the first large-scale experimental demonstration of this LLM weight quantization exploit, proving its practicality across various real-world models and highlighting the current lack of defense on model-sharing platforms. The model design hinges on this PGD-based approach, where fine-tuning involves a combined malicious and clean objective to preserve utility. The outcome successfully demonstrated the attack across three scenarios: vulnerable code generation, where quantized models significantly increased vulnerable code output; over-refusal attacks, with quantized models showing refusal rates up to 39.1%; and content injection, where quantized models increased

target phrase mentions to 74.7%. They also observed that models with long-tailed weight distributions (e.g., Phi-2) were more susceptible to this attack. Datasets used for their experiments included Code-Alpaca for instruction tuning, a subset of He and Vechev’s dataset for vulnerable code generation, databricks-15k for over-refusal, and evaluation datasets MMLU, TruthfulQA, HumanEval, and MBPP. All these datasets are generally publicly available.

Zechun Liu et al. [8] aimed to address the significant quantization errors in LLMs that arise from outliers in weights, activations, and the KV cache. Their central purpose was to enhance quantization accuracy by utilizing learned rotation matrices to effectively mitigate these outliers. They solved this problem by introducing SpinQuant, a novel method that incorporates these learned rotation matrices for optimal quantized network performance. SpinQuant leverages the principle of rotational invariance within Transformer architectures to create pairs of rotation matrices from identity mappings, which can then be merged into nearby weights without altering the full-precision network outputs. These rotations are specifically designed to produce outlier-reduced distributions, thereby simplifying quantization. Their main contribution is the development of SpinQuant, the first method to employ learned rotations for outlier mitigation, demonstrably improving the performance of quantized LLMs. SpinQuant significantly reduced the accuracy gap for LLaMA-2 7B models (with 4-bit weight, activation, and KV-cache quantization) to just 2.9 points from full precision, outperforming existing techniques like LLM-QAT and SmoothQuant. The model’s design involves learning specific rotation matrices (R1 for residual, R2 for head-wise) initialized as random Hadamard matrices and optimized using Cayley SGD. After optimization, GPTQ is applied for the final weight quantization. The outcomes were impressive: SpinQuant achieved state-of-the-art performance, consistently outperforming random Hadamard rotations. For LLaMA-2 7B in a W4A4KV4 setting, SpinQuant achieved 71.2% average accuracy and a perplexity of 3.8, significantly closer to the FP16 baseline than other methods. The optimization process was efficient, taking only minutes to hours. Qualitatively, their visualizations confirmed that rotation effectively attenuates extreme values and removes outliers from activation and weight distributions. The research primarily used the WikiText2 testset for perplexity and eight zero-shot commonsense reasoning tasks for accuracy evaluation. WikiText-2 was also utilized as a calibration set for rotation optimization and GPTQ. These datasets are publicly available.

Dongwei Wang and Huanrui Yang [1] focused on enhancing the robustness of In-Memory Computing (IMC) Deep Neural Network (DNN) hardware against adversarial input and weight attacks. Their central purpose was to mitigate the inherent lower noise margin and accuracy degradation observed in analog IMC silicon. They addressed this by proposing a novel DNN training scheme that integrates measured IMC noise and aggressive partial sum quantization directly at the IMC crossbar. The solution leverages an input-splitting algorithm to divide DNN layers, computing partial sums that are then aggressively quantized to 1-bit or 2-bit values, enabling the DNNs to adapt to these lower-precision computations during training. This approach eliminates the need for high-resolution Analog-to-Digital Converters (ADCs), simplifying hardware to comparators or 2-bit ADCs, and crucially, they posited that this aggressive quantization could mask small adversarial perturba-

tions, thereby improving robustness. Their main contribution is this pioneering DNN training scheme that effectively strengthens IMC hardware against both adversarial input (e.g., PGD) and weight (e.g., bit-flip) attacks. It is noted as the first work to comprehensively investigate both attack types and successfully integrate actual IMC prototype chip results for improved adversarial robustness. The model design involves training DNNs with injected noise from IMC hardware and specifically adapting to 1-bit or 2-bit partial sum quantization. Key pre-processing includes the generation of adversarial examples via techniques like Projected Gradient Descent (PGD) for input attacks. The outcome demonstrated significant robustness improvements: for PGD input attacks, black-box adversarial accuracy increased by up to 10%, and against bit-flip weight attacks, the scheme required over 30% more bit-flips to degrade performance. The specific dataset names used for training and evaluation are not explicitly mentioned in the provided excerpts, but the context implies standard datasets for image/speech recognition, which are typically publicly available for DNN research.

Ji Lin et al. [7] introduced Activation-aware Weight Quantization (AWQ), a hardware-friendly method for low-bit weight-only quantization in LLMs. Their primary goal was to facilitate the efficient deployment of LLMs on edge devices by substantially reducing memory and bandwidth demands while preserving the model’s accuracy and generalization capabilities. They approached this problem with a key observation: not all weights are equally important. Instead of focusing on weight magnitude, they realized that salient weights should be identified by observing the activation distribution because weights corresponding to larger activation magnitudes process more crucial features. To solve this, AWQ employs a per-channel scaling method that identifies these salient channels and scales them up to reduce their relative quantization error, thereby avoiding hardware-inefficient mixed-precision implementations. The method does not rely on backpropagation or reconstruction. Their main contribution is AWQ’s ability to offer a simple yet highly effective solution for low-bit weight-only LLM compression, consistently outperforming existing methods across various benchmarks and being uniquely applicable to instruction-tuned and multi-modal LLMs due to its superior generalization. The model design centers on this per-channel scaling based on activation magnitudes, with the optimal scaling factor determined by a fast grid search. Weight clipping is also applied to minimize MSE. Coupled with AWQ, they developed TinyChat, an efficient inference framework that demonstrated over 3x speedup compared to FP16 implementations on various GPUs, even enabling Llama-2 70B deployment on mobile GPUs. Outcomes showed near-lossless performance on language modeling (WikiText-2 perplexity), visual-language tasks (VILA models), and programming/math problems (MBPP, GSM8K), where AWQ (INT4-g128) often matched original FP16 performance. TinyChat delivered 1.2-3.0x speedup over other edge inference systems. For dataset usage, AWQ utilizes a small calibration set from the Pile, and for evaluation, relies on WikiText-2, VILA datasets, MBPP, and GSM8K. All these datasets are publicly available.

Zhang et al. [4] aimed to enhance the robustness of Artificial Neural Networks (ANNs) and Spiking Neural Networks (SNNs) against adversarial attacks, addressing the vulnerability of ANNs to imperceptible input perturbations that can mislead models. They propose a novel noise injection-based training scheme (NIT), develop-

ing a likelihood ratio (LR) method to estimate gradients for both synaptic weights and noise levels during stochastic gradient descent, and an approximation (NIT-S) designed to reduce memory consumption and improve computational efficiency. Their main contribution is demonstrating that this scheme significantly improves adversarial robustness and achieves slightly better original classification accuracy compared to conventional methods like spatio-temporal backpropagation (STBP). For model design, ANNs incorporate independent standard normal noise added to each neuron’s logit output, while SNNs are constructed as a fully connected network with one hidden layer (50 neurons) using Leaky Integrate-and-Fire (LIF) neurons and cross-entropy as the loss function. Key pre-processing for experiments on the MNIST and Fashion-MNIST datasets involved randomly splitting the MNIST dataset into training, validation, and testing sets in a 7:2:1 ratio, and selecting 1000 random images for adversarial testing with perturbation limits (e.g., a maximum perturbation of 0.1 per pixel and specific step sizes for iterative attacks). The outcomes showed that NIT achieved an optimal classification accuracy of 92.3% on original MNIST samples and significantly increased accuracy under various adversarial attacks (e.g., an 81.4% accuracy under the FGSM attack compared to 42.6% for STBP), with the simplified NIT-S offering comparable robustness with improved efficiency, and a combined NIT-S+ providing a balanced performance. While the sources explicitly mention the use of the MNIST and Fashion-MNIST datasets, they do not explicitly state whether these datasets are publicly available for research purposes.

Dongwei Wang and Huanrui Yang [14] aimed to resolve the challenge of "sensitive" or "outlier" weights in LLM quantization, which typically require special high-precision treatment, hindering efficient hardware deployment. Their primary purpose was to enable these outlier weights to be quantized to the same low bit-width as other weights without compromising model performance. Their solution, Noise Perturbation Fine-tuning (NPFT), is an efficient fine-tuning method that reduces the loss Hessian trace concerning these outlier weights. They identified outliers and introduced random weight perturbations onto them during a Parameter-Efficient Fine-Tuning (PEFT) process. NPFT estimates the Hessian trace by evaluating the expected model loss under these random perturbations, thereby avoiding the computationally expensive direct higher-order gradient calculations. Their main contribution is NPFT’s ability to efficiently reduce the sensitivity of outlier weights, allowing their uniform quantization and significantly improving quantized model performance across various post-training quantizers. Notably, NPFT empowered the simple Round-To-Nearest (RTN) quantizer to achieve performance comparable to GPTQ on the LLaMA2-7B-4bits benchmark. The model design involves identifying outlier positions using the Fisher matrix and a perturbation ratio, then applying channel-wise noise to these positions, ensuring zero mean. For efficiency, they integrate LoRA adapters on self-attention and MLP weights, with perturbations applied to the base model weights. The outcome demonstrated that NPFT consistently achieved stable perplexity improvements for uniform (RTN, GPTQ) and non-uniform (SqueezeLLM) quantizers. For instance, NPFT improved RTN’s perplexity on OPT-1.3B-4bits on both C4 and WikiText datasets. A key finding from their theoretical insight verification was the significant reduction in outlier sensitivity after NPFT. Their experiments involved LLaMA and OPT models, using

the C4 dataset for language modeling and calibration, and the WikiText2 dataset to evaluate performance on unseen data. Both datasets are publicly available.

Qianli Wang et al. [21] undertook a comprehensive study to investigate the unexplored impact of quantization on LLM explainability and interpretability. Their core purpose was to understand how quantization influences a model’s internal decision-making processes and the quality of explanations it generates, recognizing transparency as a critical requirement for LLM deployment. They approached this by conducting extensive experiments using three common post-training quantization (PTQ) techniques (GPTQ, AWQ, and bitsandbytes) at various bit widths, evaluating them against two explainability methods (counterfactual examples (CFEs) and natural language explanations (NLEs)) and two interpretability approaches (knowledge memorization analysis and latent multi-hop reasoning analysis). This was supplemented by a human user study to assess the subjective quality of explanations. Their main contribution is the finding that quantization can significantly and unpredictably affect model transparency, with the impact varying based on the quantization method, the specific explainability/interpretability approach, and the evaluation metric. They revealed that quantization generally leads to information loss, which is more pronounced in smaller models, but surprisingly, lower-precision LLMs can sometimes outperform higher-precision ones, and a quantized model might even generate better explanations than its full-precision counterpart. The study’s model design involved comparing full-precision LLMs with their quantized versions, utilizing existing tools like FIZLE for CFEs and ZeroCoT for NLEs, and specific methodologies for knowledge memorization and latent multi-hop reasoning. The outcomes were multi-faceted: Automatic evaluations showed quantization generally degrades CFE quality, especially fluency, and impacts knowledge memorization, with most information loss concentrated in the final two layers. The human user study indicated that full-precision models’ explanations were perceived as slightly more trustworthy and coherent. Across methods, integer quantization (bitsandbytes-8) consistently yielded higher-quality explanations. The datasets central to this research included eSNLI for NLEs and CFEs, LRE for knowledge memorization, and TwoHop-Fact for latent multi-hop reasoning. Additional experiments used AG News and HealthFC. All these datasets are generally publicly available.

Linyang Li et al. [3] introduced a novel watermarking strategy that embeds watermarks into the quantization process of LLMs. Their central purpose was to safeguard model weights and prevent malicious or unlicensed use of open-source LLMs, thereby enabling LLM providers to assert ownership. They achieved this by designing watermarks that are visible when the model operates in full-precision (FP32) mode but remain hidden when the model is quantized to INT8. This means users can utilize the quantized model normally for inference, but any attempt to fine-tune or use a different quantization strategy in full-precision would reveal the watermark. The authors’ main contribution is being the first to integrate quantization strategies into the LLM watermarking domain. Their proposed ”interval optimization” method allows the embedding of both text-agnostic and text-related watermarks into open-source LLMs like GPT-Neo and LLaMA. This approach offers more control than trigger-based methods (as the quantized model is watermark-free) and is more practical than decoding-specific methods. The model design hinges on the

”interval optimization” method, which exploits the computational gap between full-precision and quantized representations to embed watermarks while preserving the quality of text generated by the quantized model. They used INT8 quantization as the target and implemented a multiple-random-test strategy for verifying the watermark’s success. The outcome demonstrated successful watermark planting in both text-agnostic (always active in FP32) and text-related (activated by specific inputs) scenarios, achieving high success rates and maintaining normal text generation in quantized mode. They noted that larger models like LLaMA-7B were more challenging to watermark than smaller ones like GPT-Neo-2.7B. While further pretraining could remove watermarks, the core concept remains promising. For datasets, they utilized a subset of the Wiki corpus (specifically contexts from SQuAD) for trigger construction and a mix of PubMed and WritingPrompts for general training data. All referenced datasets are generally publicly available.

Ting-Yun Chang et al. [15] investigated the fundamental question of why specific inputs lead to disproportionately large quantization errors in low-bit LLMs. Their core purpose was to provide a mechanistic understanding of these errors and identify the critical model components responsible for maintaining performance under quantization. They approached this by analyzing various 3-4 bit weight-only quantization methods (GPTQ, AWQ, NF, EQAT) across a range of LLM sizes (7B-70B). They found strong correlations between quantization errors across different methods, indicating that they tend to fail on the same inputs. To explain this, they developed a hypothesis linking smaller residual stream magnitudes to error amplification through layers, contrary to prior beliefs that large activations were solely detrimental. They then applied LLM localization techniques, including early exiting and cross-model activation patching, to pinpoint the source of these errors. Their main contribution is providing a mechanistic explanation for large quantization errors, identifying the MLP gate projections as crucial components for perplexity preservation. They also revealed that long-tail examples tend to experience less degradation after quantization than other examples, challenging prior assumptions. The model design for their analysis involved applying standard quantization methods to various LLMs and then meticulously analyzing their internal states. Key analytical techniques included quantifying example-level quantization error via NLL differences, measuring residual magnitudes across layers, and using early exiting to assess hidden state readiness. Crucially, they employed cross-model activation patching (CMAP) to isolate the impact of specific activation types (e.g., MLP gate outputs). The outcomes showed: High correlations (average $\rho = 0.82$) in errors across different quantization methods, with significant overlap in large-error examples. Smaller residual magnitudes in upper layers strongly correlated negatively with larger quantization errors. Patching MLP gate outputs substantially reduced perplexity errors, while MHA patching had minimal effect. Surprisingly, long-tail examples (low NLLs) exhibited smaller quantization errors, and models showed the lowest degradation for the lowest and highest popularity examples on factual knowledge tasks. The research primarily used a 10k example dataset from the FineWeb corpus for NLL evaluations and PopQA for studying factual knowledge and entity popularity. Calibration sets for the quantization methods often used C4 or RedPajama. All these datasets are publicly available.

Xing Hu et al. [19] studied performance loss in post-training quantization of LLMs. They focused on how uneven data distributions and outliers affect quantization accuracy. Their main goal was to create a systematic method to improve the fit of model parameters and activations during quantization. They introduced a new metric, Quantization Space Utilization Rate (QSUR), to assess how well transformed data fits within the quantization space. Building on this, they proposed OSTQuant. This framework uses learnable orthogonal and scaling transformation pairs to globally reshape distributions without changing network outputs. To tackle optimization with limited calibration data, they developed KL-Top loss. This approach uses top-k logit alignment to maintain semantic richness and minimize overfitting. Their findings showed that OSTQuant achieves top performance across various quantization settings, including the difficult W4A4KV4 configurations. It outperformed previous methods like GPTQ and QuaRot while ensuring high efficiency and low memory use.

Jungwoo Park et al. [20] looked into whether activation outliers in large language models (LLMs) are a result of their design or a result of training methods. Their main goal was to create a scalable pre-training framework that stops these outliers from forming at the start. This approach allows for efficient low-bit quantization. They developed the Outlier-Safe Pre-Training (OSP) framework, which includes the Muon optimizer, Single-Scale RMSNorm, and learnable embedding projections. They significantly reduced outlier behavior without changing the model architecture or adding major computational costs. The study showed that models trained with OSP had nearly zero excess kurtosis and were much more stable under heavy 4-bit quantization, scoring an average of 35.7 compared to 26.5 for models trained with Adam. Additionally, the research challenged earlier beliefs by demonstrating that attention sinks, which were thought to create outliers, can exist without causing large activations. These findings changed the understanding of quantization errors in LLMs, suggesting that outliers can be avoided through careful training design.

Mohammad Mozaffari et al. [12] introduced SLIM, a one-shot compression framework designed to lower the computational and memory costs of large language models (LLMs) without sacrificing performance. The main issue it addresses is how to compress LLMs effectively while maintaining performance, especially with structured sparsity constraints like NVIDIA’s 2:4 pattern. SLIM incorporates three main parts: a probabilistic uniform quantization approach (SLIM-Quant), a sparsification step using existing one-shot pruning techniques, and a new saliency-based low-rank adapter module (SLIM-LoRA). Unlike traditional methods that need retraining, SLIM calculates the best low-rank adapters analytically using an additive and invertible saliency function based on weight-activation interactions. The authors showed that SLIM outperforms leading one-shot methods such as SparseGPT and L2QER in both structured and unstructured sparsity conditions, achieving up to 5.66% higher accuracy on LLaMA-2-7B with 2:4 sparsity and 4-bit quantization. Additionally, SLIM reached up to a $4.3\times$ speedup and a $0.23\times$ memory reduction, while also providing optional lightweight fine-tuning with quantized low-rank adapters to further narrow the gap with dense models. The results highlight that high-performance compression is possible without costly retraining and suggest that saliency-aware methods can be essential in balancing efficiency and accuracy for real-world LLM deployment.

Dettmers et al. [2] looked at quantization in terms of performance and accuracy, but their findings also offered insights for adversarial robustness. Their main goal was to reduce the memory usage of large language models (LLMs) by compressing weights to 3 to 4 bits while keeping nearly the same accuracy. To do this, they introduced SpQR, a hybrid representation that separates outlier weights causing significant quantization errors and stores them in higher precision while compressing the others. A key takeaway from their work was the discovery of structured patterns of quantization sensitivity in LLMs, including row and column outliers and artifacts specific to attention heads. These patterns informed their efficient compression approach and highlighted vulnerabilities that could be exploited in adversarial situations. Although their research was not intended as a security paper, it emphasized the need for quantization-aware model design and demonstrated that quantization sensitivity varies widely, making some parts of a model significantly more fragile than others.

Brian Chmiel et al. [17] examined the important question of whether large language models can be fully trained using ultra-low-precision formats, specifically FP4. Their main goal was to show that end-to-end FP4 training can work across weights, activations, and gradients, which were thought to be unstable at such low bit widths. They ran extensive experiments to find the best block sizes and scaling formats, eventually identifying NVFP4 (E2M1 data with E4M3 scale and block size 16) as the most effective setup. They also looked into quantization stability and introduced a split rounding strategy: stochastic for backward/update and round-to-nearest for forward, to stabilize training. Importantly, they found a theoretical limit where gradient magnitudes become too small for FP4 to be effective and suggested quantization aware fine-tuning as a solution. Their results showed that FP4 training, when set up correctly, performs as well as BF16, confirming its practical use for large-scale LLMs.

Ma et al. [9] studied the rise of hidden security threats in commercial post-training quantization workflows. Their main goal was to look into how inactive backdoors could be added to float-32 models and later triggered only through quantization with common frameworks like TensorFlow Lite and PyTorch Mobile. They introduced the "PQ backdoor" concept, where a backdoor stays completely inactive in the full-precision model, even when triggers are present, but activates with high success rates once the model is quantized. The study revealed that four prominent backdoor detection systems Neural Cleanse, STRIP, ABS, and MNTD were unable to find the inactive backdoors. This failure makes traditional inspection tools ineffective. The authors also showed that this vulnerability exists across different datasets and architectures, exposing a serious flaw in current deployment practices. A key finding was the demonstration that even industry standard frameworks could unknowingly support adversarial behavior after quantization. They suggested that injecting Gaussian noise might help partially, but it is not a complete solution.

Hasan [6] has described the methods of quantization techniques to optimize Large Language Models (LLMs), with the main goal being Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). The researchers tested many dense-

neural network structures of different sizes using synthetic datasets that are meant to match real-world LLM tasks. This study showed that quantization, especially in the case of introduction of adaptive scaling factor, could cut the model size by up to 68 percent and the computation cost by up to 60 percent without changing the performance to more than six percent of full-precision baselines. Performance was measured with the use of such metrics as perplexity, BLEU score, inference latency, and power consumption. The results suggest that quantization is also relevant when deploying LLMs in low-resource settings since it can provide a high level of efficiency with minimal accuracy loss.

Melin et al. [11] compared the code generation capabilities of smaller open-source Large Language Models (LLMs) systematically and analysed the effect of quantization to 8-bit and 4-bit on the same. Based on benchmarked models including HumanEval Plus and MBPP Plus, in addition to metrics including code bleu and static analysis through SonarQube, the study revealed all the models had problems with functional correctness and the best benchmark pass rate was 25%. Quantization turned out to be mixed: on one hand, it helped some models to perform better on simpler tasks; on the other hand, it negatively impacted the performance of others. Moreover, the generated codebase had more than 21 percent of maintainability and reliability problems. By drawing conclusions, the authors suggested that applying quantization allows LLMs to be more resource-friendly to their users, but LLM-based code frequently underperforms, which motivated the necessity of a thoughtful assessment prior to the use of LLM-generated code in practice.

Malinovskii et al. [10] compared and contrasted the theory and practice of large language model (LLM) quantization, where they developed and proposed the so-called linearity theorem, which was formally used to relate per-layer quantization errors to the overall model performance. In experiments to evaluate their approaches on standard benchmarks including WikiText-2 and various zero-shot and few-shot reasoning tasks, the authors also suggested the HIGGS quantization technique which was data-free/hardware-friendly since it used Hadamard rotations to achieve the best quantization. They showed in their Llama-3 and Qwen experiments that HIGGS retained superior performance to the previously published data-free quantization methods across the 3, 4 bits regime, and was able to reach perplexity and accuracy comparable to full-precision baselines at a much higher level of computational efficiency. The work is an in-depth introduction and a useful practical tool to a resource-efficient LLM deployment, in particular, in resource-limited settings.

Proskurina et al. [13] examined the effects of post-training quantization, i.e., utilization of the GPTQ method, on the confidence and calibration of large language models (LLMs) including BLOOM, OPT, Mistral-7B, and LLaMA-7B. Testing on a number of commonsense reasoning and question-answering benchmarks, the study discovered that 4-bit quantization tended to replace calibration error with an overall decrease in model confidence, especially in the predictions where the original model tended toward skepticism. Although the overall accuracy losses were not very big, the confidence estimates' credence was seriously undermined, particularly in minor models. The quantized LLMs will possibly need extra adjustment before they can be assured of reliable output in sensitive issues, the authors concluded.

Chen et al. (2025) [16] carried out an in-depth evaluation of the safety risks that come with quantization in Large Language Models (LLMs). They specifically looked into the balance between how well the model performs and its safety alignment, especially in environments where resources are limited. Their research examined popular quantization methods, including Post-Training Quantization (PTQ) techniques like AWQ and AQLM, as well as Quantization-Aware Training (QAT) methods such as LLM-QAT and QLoRA, across different bit-widths (INT4, INT8). The authors found that quantization tends to weaken safety features, with lower bit-widths (like INT4) leading to much higher Attack Success Rates (ASR) compared to models that use full precision. They also discovered that calibration datasets, which are often used to maintain utility, can unintentionally increase these safety risks if they include even indirectly harmful samples. To address these vulnerabilities, the authors introduced Q-resafe, a framework designed for quantization-aware safety patching. This approach uses a safety-patching dataset created through knowledge distillation from the model before quantization and applies Direct Preference Optimization (DPO) to realign the quantized model. A standout feature of their method is the identification and selective updating of "safety-critical weights" using SNIP scores, which helps the model restore its safety mechanisms while keeping most parameters unchanged to maintain utility. Their experimental results indicated that Q-resafe effectively realigned quantized models to nearly the same safety levels as pre-quantization, all while keeping computational demands low.

2.2 Summary of Key Findings

The review of the systematic literature highlights three key patterns that drive the research on Anchor-Guided Repair: the rise of quantization as a new attack vector, the significant effect of outlier anchor weights on model stability, and the shortcomings of current defense strategies against weight noise.

2.2.1 Quantization as a Security Vulnerability

A major trend in recent studies is the transition from seeing quantization simply as a compression method to recognizing it as a vital security concern. While traditional techniques like GPTQ and AWQ aim to maintain utility, they unintentionally create vulnerabilities in compromised models.

Dormant Backdoors: Research by Egashira et al. [5] and Ma et al. [9] shows that quantization can be manipulated to develop sleeper agents. Malicious actions can be concealed within the full-precision (FP32) model, appearing harmless until the model is quantized (for instance, to INT4), at which point the malicious payload activates. This directly supports the thesis’s focus on Compromised Large Language Models.

Safety Degradation: Chen et al. [16] and Proskurina et al. [13] reveal that aggressive quantization, especially at 4-bit, inherently undermines safety alignment and confidence calibration, resulting in higher Attack Success Rates (ASR) and issues like overrefusal or vulnerable code generation.

2.2.2 Weight Sensitivity and the “Anchor” Hypothesis

The literature strongly supports the thesis premise that not all weights are equal; specific anchor weights dictate model stability.

Outlier Dominance: Czakó et al. [18] and Chang et al. [15] identify that quantization errors are not uniformly distributed but are driven by activation outliers and specific components like MLP gate projections.

Saliency & Sensitivity: Lin et al. [7] (AWQ) and Dettmers et al. [2] (SpQR) prove that isolating salient or outlier weights and keeping them in high precision is essential for performance. Wang and Yang [14] further show that “sensitive” weights require targeted noise perturbation fine-tuning (NPFT) to prevent degradation.

Implication for Thesis: These findings lay the theoretical foundation for what we call Anchor-Guided Repair. If only a small number of weights—referred to as anchors—are responsible for how sensitive the model is to quantization, then focusing on repairing or stabilizing these specific anchors is a much smarter defense strategy compared to retraining the entire model.

2.2.3 Robustness Against Weight Noise and Perturbations

Recent studies point out a significant gap in our ability to defend against weight noise attacks, which are essentially created during the quantization process

Noise Injection Training: Research by Wang and Yang [1] and Zhang et al. [4] shows that incorporating injected noise during training can boost resilience against adversarial inputs and bit-flip attacks. However, these techniques are typically used in the pre-training phase or during standard fine-tuning.

The Repair Gap: While Chen et al. [16] introduce Q-resafe to help realign models through preference optimization, and Liu et al. [8] implement SpinQuant to adjust matrices for outlier management, there’s a noticeable absence of post-training repair strategies specifically aimed at stabilizing models that have already been compromised by the dual challenges of low-precision conversion and weight noise.

2.2.4 Conclusion and Correlation with Thesis Objective

Findings of existing research clearly shows that quantization can trigger hidden vulnerabilities, and the stability of a model relies on a few sensitive weights, often referred to as anchors. Yet, most current approaches mainly aim to maintain accuracy or identify backdoors, rather than focusing on restoring stability when a model is compromised.

This thesis addresses this critical gap by proposing Anchor-Guided Repair. By identifying the sensitive “anchor” weights highlighted by Lin et al. [7] and Chang et al. [15], and applying targeted repair mechanisms resilient to the noise identified by Wang and Yang [1], this research aims to immunize LLMs against the specific quantization exploits demonstrated by Egashira et al. [5].

Chapter 3

Requirements, Impacts and Constraints

The chapter describes the technical specifications, social and environmental effects, ethics, guidelines, project management strategy, risk and economic viability of the suggested Anchor-Guided Repair defence mechanism to stabilize the damaged pre-trained language models.

3.1 Final Specifications and Requirements

Hardware Requirements

- **CPU:** Intel Core i7-14700K (20 cores / 28 threads), which is utilized in data preprocessing, tokenization, and evaluation and orchestration.
- **GPU:** NVIDIA RTX 4080 Super (16 GB VRAM), which is employed in fine-tuning of models, defense training, and perplexity evaluation.
- **Memory:** 64 GB of DDR5 RAM, which was not insufficient to run model checkpoints of large size, loading datasets, or to perform an evaluation at prompt levels.
- **Storage:** 1 TB SSD, which is used to store trained models, compromised and defended checkpoints, evaluation logs, and result files.
- **Power Supply:** 850W PSU, which supports a stable performance of the graphics card and system during long training and testing sessions.

This hardware design fosters the effective experimentation in large language models, makes it possible to train stable FP16 or conduct several defense and evaluation runs without resource bottlenecks.

Software Requirements

Operating System: Windows 10/11 (64-bit) that offers a predictable development environment to Python-based machine learning workflows and CUDA acceleration via the use of a graphics card.

Python Version: Python 3.10 or 3.11 (64-bit) compatible with the rest of the deep learning ecosystem such as PyTorch 2.x and Transformers and with the newest language features utilized in the implementation.

Deep Learning Framework: PyTorch 2.0 or higher with support of CUDA 11.8 or 12.1 is the main deep learning framework used to train models, defense fine-tune models, and evaluate the perplexity of various precision modes (FP16, INT8, 4-BIT).

Machine Learning Libraries: Hugging Face Transformers 4.35+, includes the AutoModelForCausalLM, AutoTokenizer, Trainer, TrainingArguments, and DataCollatorForLanguageModeling modules needed to load pre-trained LLMs (Qwen2.5, Pythia), tokenize, and fine-tune.

Hugging Face Datasets 2.14+, used to load and process the WikiText-2 benchmark dataset to evaluate the perplexity and train the defense.

BitsAndBytes 0.41+, which allows INT8 and 4-bit quantization of large language models by the BitsAndBytesConfig integration with the Transformers library.

Scientific Computing: NumPy 1.24+, the library used to perform numeric operations, statistical computation and sigma calibration of noise injection attacks.

Data Visualization: Matplotlib 3.7+ used to generate the plots of defense effectiveness, perplexity comparison bar charts, visualizations of recovery rates, and category heatmaps to be analyzed and published.

Progress monitoring: tqdm 4.65+, offers progress bars to long-term computations and operations such as tokenization, iteration of perplexity calculations, and training cycles.

CUDA Toolkit: The NVIDIA CUDA Toolkit 11.8 or 12.1, that offers the GPU acceleration runtime to PyTorch operations, such as model inference, gradient calculation and quantized model execution.

Development Environment: Visual Studio Code which act as the integrated development environment of code editing, debugging, and terminal execution.

Git 2.40+ which is used to control and track model checkpoints, experiment settings and result files.

Package Manager: pip 23.0+ or conda 23.0+ which are used to handle Python package dependencies and build isolated virtual environments to allow reproducible experiments.

Data Requirements

- WikiText-2 corpus, separated into training, validation and test parts, offers the necessary data base.
- A test battery of 96 prompts based on the extensive range of task typologies is used:

- **Code Generation**
- **Mathematical Reasoning**
- **Question Answering**
- **Summarization**
- **Creative Writing**
- **Logical Reasoning**
- **Translation**
- **Sentiment Analysis**
- **Information Extraction**
- **Instruction Following**
- **Common Sense Reasoning**
- **Domain Knowledge (Science)**

Model requirements

- Pretrained language models, such as Pythia-410M and Qwen2.5-0.5B
- The capacity to retrieve and work with model weights
- Assistance with FP16 assessment and adjustment

3.2 Societal Impact

The defense mechanism suggested can improve the reliability and trust of the open-source language models.

Positive Impacts

- It shields users against the unintentional deployment of compromised or degraded models.
- It enhances the strength of AI systems that are used in:
 - Education
 - Research
 - Software development
- It helps organizations to identify and correct model corruption without having to train afresh.

Broader Implications

- Promotes the safer repurposing of trained models.
- Encourages the ethical application of AI in settings with limited resources.

3.3 Environmental Impact

The design of the proposed method is computationally friendly.

Sustainability Benefits

- Does not retrain large language models completely.
- Short fine time schedules (under 200 steps) are used.
- Saves on energy used in comparison to complete pretraining.

Environmental Considerations

- The use of GPU is a contributor of energy.
- **Mitigated by:**
 - Single-GPU training
 - Small batch sizes
 - Short training duration

On the whole, the strategy advances energy-saving model mending.

3.4 Ethical Issues

Ethical Considerations

- The defense mechanism does not bring new information.
- No personal, sensitive and user-generated data is being used.
- It uses only public datasets (WikiText-2).

Responsible Use

- Model behavior should be manipulated, not repaired.
- Does not cause any change in model intent or bias.
- Promotes openness in the assessment of models and strength.

3.5 Standards

Although there is no hard and fast industry rule applied to model repair, this work is in line with:

- Reliability and reproducibility of software by IEEE.
- Guidelines of secure data management and accountable artificial intelligence.
- Good machine learning research practices.

3.6 Project Management Plan

Development Timeline

- Phase 1: Preparation and evaluation of baseline model.
- Phase 2: Simulation of noise based attack.
- Phase 3: Training in anchor guided defense.
- Phase 4: Testing on WikiText-2 and 96-prompt suite.
- Phase 5: Analysis, visualization and reporting.

Resource Allocation

- Individual researcher (training, evaluation, analysis)
- GPU-based experimentation
- Trial and error testing and parameter optimization.

3.7 Risk Management

Identified Risks

- Sensitivity based on architecture (e.g., Pythia vs. Qwen)
- During defence training, overfitting

Mitigation Strategies

- Strict numerical checks and FP16-safe evaluation
- Variance analysis and prompt-level sanity checks
- Bounded recovery constraints and early stopping

These tactics guarantee consistent and trustworthy assessment.

3.8 Economic Analysis

Analysis of Costs

- Cloud services are not necessary.
- Stack of open-source software.
- One GPU suitable for consumers.

Cost estimate:

- GPU hardware (one-time).
- Low electricity costs as a result of brief training sessions.

Estimating Benefits

- Prevents expensive retraining of big models.
- Permits the reuse of compromised models.
- Conserves financial and computational resources.

The suggested defence is both affordable and useful in practical situations.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The following section outlines the technical architecture and algorithm of the proposed defense mechanism, including the dual-model anchoring strategy, loss functionalization, parameter selection criteria, and multi-precision evaluation protocol that can hold the performance recovery controlled under weight-noise attacks with limited and understandable recovery measures.

As outlined in Figure 4.1, the experimental pipeline proceeds through five distinct stages: starting with baseline establishment, moving to task adaptation, simulating the weight attack, training the anchor-guided defense, and concluding with a multi-metric evaluation.

4.1.1 Baseline Establishment and Initial Performance Assessment

The establishment phase performs a thorough initial assessment of the trained language model under a variety of precision settings in order to determine the zero-shot performance properties of the model and measure the effect of post training quantization on model quality. The pretrained model (Qwen2.5-0.5B-Instruct or Pythia-410M) is loaded directly using its original checkpoint with no fine-tuning or adaptation and perplexity is measured on the WikiText-2 test split at three levels of precision, including FP16 (half-precision floating-point), INT8 (8-bit integer quantization with per-tensor symmetric scaling), and 4-BIT (NormalFloat 4-bit quantization with BitsAndBytes NF4 format).

This multi-precision test demonstrates that the sensitivity to quantization of the pretrained weights will typically achieve a 5-15 percent perplexity increase when switching to INT8 precision and 15-30 percent when switching to 4-BIT precision, which will be explained by simply being caused by having lower numerical precision, and not by adversarial corruption.. The standardized test evaluation benchmark across all the experimental stages is the WikiText-2 test set, which consists of a variety of Wikipedia articles across different domains and writing styles, and makes it possible to guarantee the consistency of measurements and directly compare different model states (clean, attacked, defended).

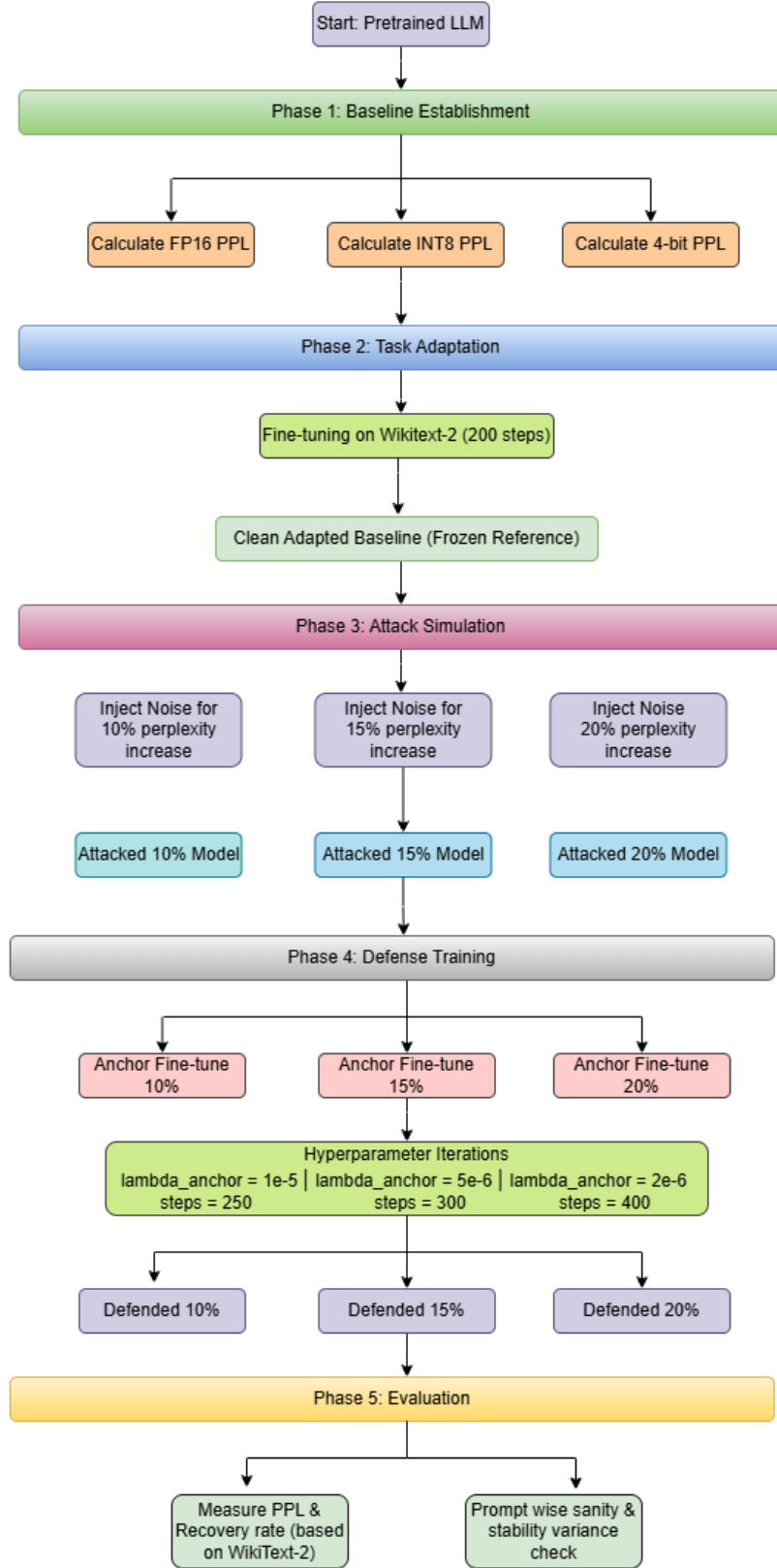


Figure 4.1: Experimental Methodology Flowchart.

These baseline measurements form important reference points to further analysis in that they determine the performance properties of the initial unaltered pretrained model and measure the independent influences of quantization-induced errors prior to introducing the effects of any form of task adaptation or adversarial perturbation. The phase of establishment also has the diagnostic value of checking model integrity, ensuring that tokenizers, evaluation scripts, and quantization pipelines are working, and determine deterministic evaluation protocols with fixed random seeds (seed=42) to get the same result when repeated experiments are done. Although these zero-shot baseline measurements are taken to be complete, they are not considered in calculating the recovery rates during Phase 1 in favor of the task-adapted clean baseline that will be developed in Phase 2 since recovery rates using defended models versus zero-shot references will generate methodological artifacts that conceal the actual defense rates.

4.1.2 Task Adaptation and Clean Baseline Establishment

The task adaptation phase helps to overcome a methodological issue at the core by defining a reasonable level of performance by fine-tuning the pretrained model on the WikiText-2 system, and thus remove the confounding impact of zero-shot evaluation which may result in nonsensical recovery measures. The trained model (Qwen2.5-0.5B-Instruct or Pythia-410M) is supervisedly fine-tuned on WikiText-2 training split using common causal language models objectives without any anchoring constraints ($=0$), and where the actual aim is to adapt the internal representations of the model to the statistical properties and language structure of the test domain.

The training program takes 200 optimization steps with AdamW optimizer and learning rate $2e-5$, batch size 1, and gradient accumulation 4 steps (effective batch size 4) on the training corpus of WikiText-2 (which has been concatenated and broken down into 2048-token sections by the same preprocessing protocol) with this choice. This adaptation stage is able to gain any significant reduction in perplexity on the WikiText-2 test split (is often between 15-18 and 10-12 on zero-shot evaluation versus adapted models), producing a task-specific performance ceiling, which is the most optimal performance that can be achieved, given the model architecture and dataset properties.

The resultant task-adapted clean baseline is two-fold in the pipeline of the experiment:

1. Since the basis of creating attacked models by controlled weight-noise injection is to ensure that the adversarial perturbations are done to weights that already hold task-relevant information instead of zero-shot representations.
2. As the frozen anchor reference during defense training, providing stable gradient signals that guide the optimization process toward recovering task-adapted knowledge rather than attempting to simultaneously learn task adaptation and noise removal.

Upon completion of fine-tuning, the adapted model is saved in FP16 format with all optimizer states and training checkpoints preserved, and undergoes comprehensive

evaluation at FP16, INT8, and 4-BIT precision levels to establish clean baseline metrics that will serve as the upper performance bound for all subsequent recovery calculations. This methodological refinement ensures that the experimental framework produces bounded and interpretable recovery rates ($\leq 100\%$), as defended models that undergo additional fine-tuning are fundamentally constrained by the performance ceiling established by this task-adapted clean baseline rather than being compared against artificially weak zero-shot references.

4.1.3 Attack Simulation Framework

Motivation

In order to model adversarial degradation and test model robustness, a weight-noise attack is proposed in this thesis. In contrast to adversarial input perturbations, this attack mitigates the internal weights of the model directly, which is used to denote quantization-based corruption or malicious parameter manipulation.

Attack Formulation

Let W be a trainable parameter tensor. The attacked parameter tensor W' is defined as:

$$W' = W + \epsilon$$

where ϵ is sampled element-wise from a Gaussian distribution with mean 0 and variance σ^2 :

$$\epsilon \sim \mathcal{N}(0, \sigma^2 I)$$

The parameter σ controls the strength of the attack.

Measuring Attack Severity via Perplexity Increase

Let PPL_0 be the perplexity of the clean baseline model and $PPL(\sigma)$ be the perplexity after applying noise with standard deviation σ . Define the degradation ratio as:

$$r(\sigma) = \frac{PPL(\sigma)}{PPL_0}$$

Attack severity levels are defined by target ratios:

- 10% attack: $r = 1.10$
- 15% attack: $r = 1.15$
- 20% attack: $r = 1.20$

The graphs in Figure 4.2 demonstrate the widening performance gap between the Clean Baseline (solid line) and the Attacked Model (dashed line) as noise intensity increases. Notably, the degradation is most severe in the 4-BIT quantization regime at 20% noise. On the other hand, in Figure 4.3, Pythia-410M exhibits high sensitivity to weight noise. The sharp ascending bell curve of the perplexity of the attacked model in the 4-BIT scenario implies that low-precision quantization greatly increases the harm due to the noise injection.

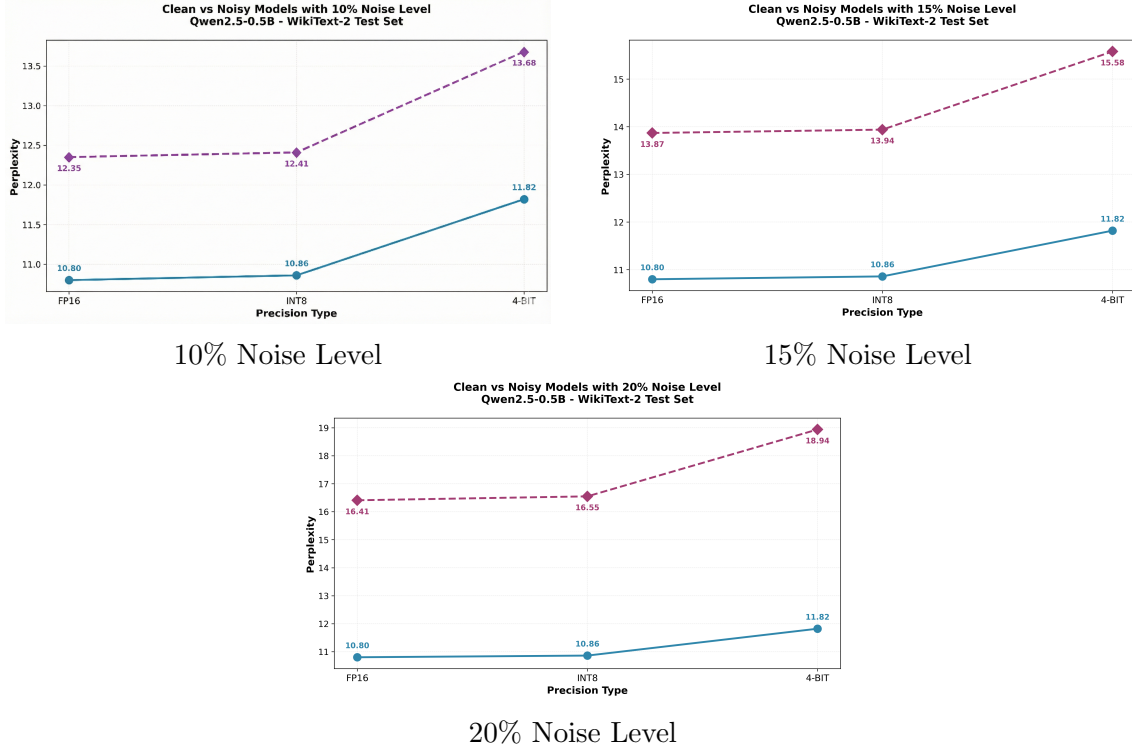


Figure 4.2: Impact of Gaussian Weight Noise on Perplexity Degradation across Varying Noise Intensities (10%, 15%, and 20%) for Qwen2.5-0.5B.

Calibrating Sigma to Achieve Target Attack Levels

Experimental data show that the dependence of the noise variance on the response of the model takes an approximately quadratic pattern when plotted in a logarithmic scale which we can see it in Equation 4.1. Based on this, the function $r(\sigma)$ is to be modeled as a second-order polynomial in σ with the coefficients a and b being estimated by regression on the validation data.

$$\log(r(\sigma)) \approx a\sigma^2 + b \quad (4.1)$$

The response function can be directly expressed in exponential form by exponentiating the fitted logarithmic expression like in Equation 4.2. This representation preserves the observed nonlinear behaviour and is useful for estimating the response at various noise levels. This can be roughly expressed as:

$$r(\sigma) \approx \exp(a\sigma^2 + b) \quad (4.2)$$

By reversing the fitted response model, the corresponding noise level σ^* can be calculated given a predetermined target response, r_{target} which can be seen in Equation 4.3. This makes it possible to directly control the amount of noise needed to produce the desired system behaviour.

$$\sigma^* = \sqrt{\frac{\log(r_{target})}{a}} \quad (4.3)$$

The coefficient a is obtained via linear regression over multiple measured pairs of

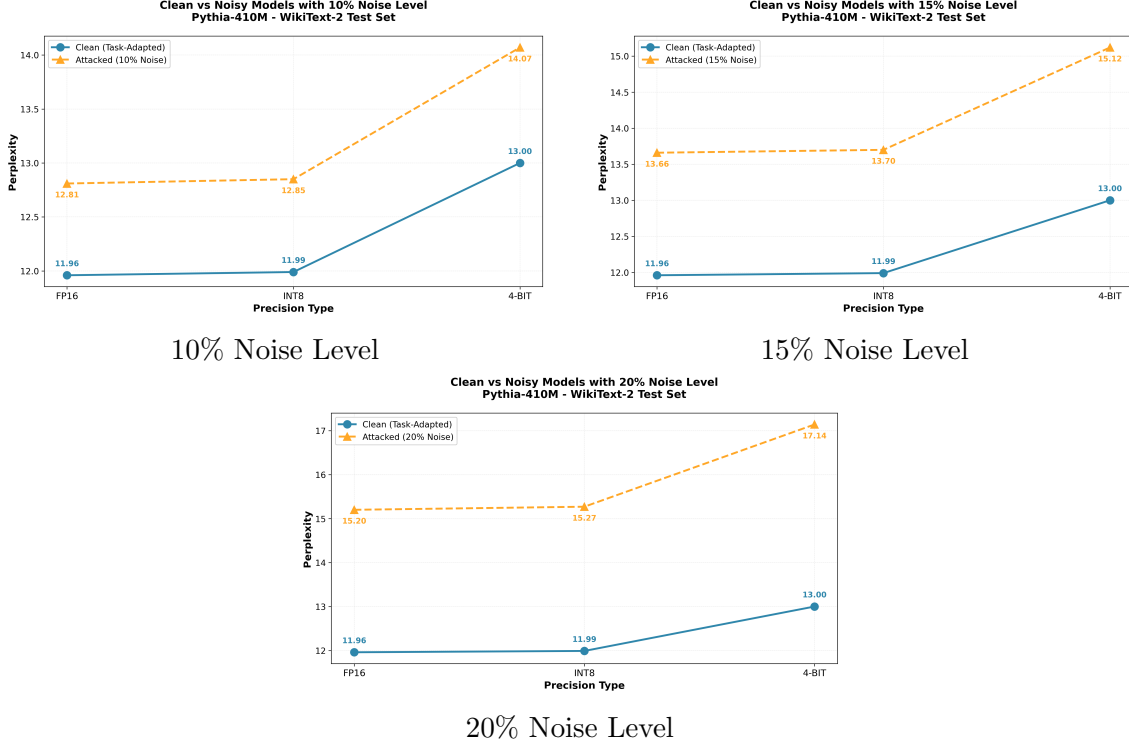


Figure 4.3: Impact of Gaussian Weight Noise on Perplexity Degradation across Varying Noise Intensities (10%, 15%, and 20%) for Pythia-410M.

$(\sigma^2, \log(r))$. This calibration ensures that attack severity is comparable across different models and precision formats.

4.1.4 Defense Training Through Anchored Fine-Tuning

The defense training phase also applies the dual model architecture that is optimized for the task-adapted clean base-memory efficiency and computational stability. The task-adapted clean base-execution speed is determined by the computation being done. The baseline model is used as a frozen anchor reference that is stored in the memory of the CPU, while a replica of the model that was attacked is loaded as a trainable instance on the GPU. memory. The anchoring mechanism selectively targets 168 critical weight matrices distributed across the model’s 24 transformer layers, specifically including the attention projection matrices (q_proj, k_proj, v_proj, o_proj) and MLP feed-forward projection matrices (gate_proj, up_proj, down_proj), while deliberately excluding normalization layers, bias parameters, and embedding matrices that primarily serve structural rather than semantic functions.

During each training iteration, the system processes a 2048-token chunk from the WikiText-2 training dataset through a four-step computational pipeline:

1. Forward pass through the trainable model to generate next-token predictions and compute standard cross-entropy loss (L_{CE}).
2. Parameter-wise L2 distance calculation between current trainable weights and their corresponding frozen anchor weights, with computation performed on

CPU to conserve GPU memory.

3. Aggregation and averaging of L2 penalties across all 168 anchored parameters to produce the anchor loss (L_{anchor}).
4. Construction of the combined objective function $L_{total} = L_{CE} + \lambda \times L_{anchor}$ where $\lambda = 1e - 5$ controls the regularization strength.

The optimization process executes 200 training steps using the AdamW optimizer with learning rate 2e-5, gradient accumulation over 2 steps (effective batch size of 2), and linear warmup over 10 steps, updating all 494 million parameters in half precision (FP16) while the anchoring constraint guides weight updates toward a balanced state that minimizes prediction error on clean data while preventing excessive deviation from the clean baseline. Upon completion of the training cycle, the defended model is saved in FP16 format, preserving the recovered weight configuration for subsequent quantization and evaluation.

4.1.5 Multi-Precision Evaluation and Performance Analysis

The evaluation stage is based on a multi-precision testing program that evaluates the effectiveness of the defense against three quantization configurations under various deployment conditions and resource limitations. The defended model is sequentially evaluated with FP16 precision (half-precision floating-point, model size of about 988 MB), INT8 precision (8-bit integer quantization via per-tensor symmetric scaling, model size of about 494 MB), and 4-BIT precision (NormalFloat 4-bit quantization using BitsAndBytes NF4 format, model size of about 247 MB), and the measurements of perplexity are done on the WikiText-2 test split using the same evaluation protocols across all three model states.

For each precision level, the recovery rate is computed using the formula:

$$\text{Recovery} = \frac{PPL_{attacked} - PPL_{defended}}{PPL_{attacked} - PPL_{clean_adapted}} \times 100\%$$

This gives a normalized measure, which measures the percentage of attack-related degradation that the defense mechanism is able to remedy. The critical ordering constraints are checked and proved to be true such that $PPL_{clean_adapted} < PPL_{defended} < PPL_{attacked}$ on all the precision settings and the fact that the defended model recovers meaningfully without necessarily achieving more than the task-adapted clean baseline and the meaning by which the recovery is achieved is limited ($\leq 100\%$).

In addition to aggregate measures of perplexity, the analysis also uses prompt-wise analysis through a curated set of 96 prompts across several different task classes, which allows checking in both the ordering constraints the individual sample level and measuring the variance reduction to understand which improvements are being made by the attacked model. The percentage of degradation of both the attacked and defended models against the clean adapted one at each level of precision offer finer details about how weight noise and quantization errors interact or add, and show that the defense process does not lose its effectiveness when models are aggressively pruned on the edges to deploy them to real-world environments. The

combination of these extensive evaluation measures, as a whole, provides the practical validity of the defense mechanism by showing reproducible partial recovery rates of 60-80% and comes with interpretable performance limits and predictable behavior under a wide variety of quantization schemes.

4.2 Preliminary Design or Design (Model) Specification

The preliminary design specification establishes the technical architecture of the defense mechanism, defining the dual-model framework, mathematical loss formulation, selective parameter targeting strategy, and multi-precision evaluation protocol that collectively enable controlled performance recovery from weight-noise attacks while maintaining bounded recovery rates.

4.2.1 Theoretical Framework: Quantization and Perplexity

Quantization as a Low-Precision Attack and Its Impact on Model Quality

Quantization as a Low-Precision Attack and Its Impact on Model Quality Quantization is a model compression method that uses fewer bits of memory on a GPU or CPU to compress the neural network parameters and run faster. Rather than storing weights in floating point representations like FP32 or FP16, quantized inference stores weights in lower-precision representations like INT8 or 4-bit. Within the framework of this thesis, quantization is considered a low-precision perturbation, which can serve as an attacker instance when not implemented with resistance to it. Quantization increases the deployability and efficiency, but the approximation error is introduced to the weight tensors, which can impair the accuracy and stability of the model.

Quantization as a Weight Approximation

Let w be an original floating-point weight value. Quantization maps w to a discrete integer value q using a scaling factor $s > 0$ as follows:

$$q = \text{clip}(\text{round}(w/s), q_{min}, q_{max})$$

The quantized approximation of the weight in real-valued form is then reconstructed as:

$$\hat{w} = s \times q$$

The quantization error is defined as:

$$\delta_w = \hat{w} - w$$

This inaccuracy disturbs the calculations of linear projections and attention layers as well as feed-forward layers. Because transformer models use numerous such operations repeatedly on many layers, minor perturbations can propagate and change the probability distribution of the model output, eventually lowering the quality of language modeling.

Quantization Configurations Used in This Work

The attacks of low precision are implemented in this thesis by quantizing the weights into low-precision numbers, applied in the Hugging Face Transformers framework by the BitsAndBytes backend at three precision levels:

- **FP16 (baseline):** Weights of the model loaded in half precision (float16), are a powerful baseline that can be used to evaluate the performance of a GPU.
- **INT8:** 8-bit quantized weights of a moderate low-precision configuration.
- **4-bit (NF4):** 4-bit NormalFloat (NF4) quantization with a FP16 computation and optional double-quantization metadata compression, which is an extreme low-precision case.

These settings are applied as controlled low-precision perturbations to examine the impact of numerically diminished precision on perplexity, robustness, and recovery in the offered defense mechanism.

Why Quantization Increases Perplexity

Quantization modifies the original weight matrix W into a perturbed version:

$$\hat{W} = W + \Delta W.$$

This is a perturbation of the logits generated by the model. As the next token prediction probability is influenced by these logits using the softmax function even tiny weight error can decrease the probability of the correct token. This increases the negative log-likelihood correspondingly causing greater perplexity.

Perplexity: Definition, Interpretation, and Computation

Definition

A standard evaluation metric of language models is the perplexity (PPL), which is a measure of the ability of the model to predict a sequence of tokens. The less the perplexity, the better the predictive performance. Given a token sequence $x_1, x_2, \dots, x_N$, a causal language model factorizes the joint probability as:

$$p(x_{1:N}) = \prod_{t=1}^N p(x_t | x_{<t})$$

The average negative log-likelihood (cross-entropy loss) is:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log p(x_t | x_{<t})$$

Perplexity is defined as:

$$\text{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{N} \sum_{t=1}^N \log p(x_t | x_{<t})\right)$$

Interpretation

The perplexity may be seen as the effective number of possible choices that the

model takes into account when making predictions of each token. As an example, a perplexity k model acts like a model that picks k , on average, equally likely tokens. Since quantization/attack reduces the model quality, the probability mass no longer takes values on the correct tokens, becoming more uncertain and hence adding more to the perplexity.

Computation Used in This Thesis

Transformer models have a limited context length, which means that long sequences cannot be reconstructed in one forward pass. Hence, the sliding-window assessment approach is applied in this thesis using a Maximum window length of 2048 tokens and a Stride of 512 tokens.

A piece of the text is processed by each window. The loss is only calculated on the new tokens, and the previous ones are covered by the mask of changing the labels to -100 to make them not count on the loss. Where l_i is the cumulated negative log-likelihood of window i and N is the cumulative number of tokens considered, perplexity is computed as:

$$\text{PPL} = \exp\left(\frac{\sum_i l_i}{N}\right)$$

This approach avoids double-counting tokens and produces a stable perplexity estimate.

4.2.2 Data Sources and Evaluation Inputs

WikiText-2 Dataset

WikiText-2 is a set of textual data that is used as the main source of model adaptation, defense training, and aggregate evaluation. WikiText-2 is an assortment of outstanding English articles from Wikipedia that frequently serve as a benchmark when assessing language modelling ability. The dataset is split into predetermined training, validation, and test splits, which are the same in this study. Each data sample has one text field that has long-form natural language data and does not have explicit labels, which qualifies it to be used in causal language modeling. Text samples associated with each split are divided into tokens in preprocessing with the tokenizer belonging to the pretrained base model. Any tokenized text in a split is bunched together in a linear sequence and divided into deterministic sized blocks of 2048 tokens, which is the maximum length of context in the model. The language-modeling instances are each a block, and the task will be to predict the next token based on all the previous ones. Any number of additional tokens that do not constitute a complete block is ignored, as it is normal in language modeling.

Prompt Suite for Coordinated Assessments

While WikiText-2 offers a powerful reference point in terms of assessing overall language modeling performance, it does not enable the investigation of the model behavior on a case-by-case basis. In order to overcome this shortcoming, prompt-level robustness analysis is performed on a set of 96 pre-selected evaluation prompts. This prompt structure consists of short instruction-based inputs which are supposed to prompt certain behaviors of the model. 12 diverse activity categories are covered

by the prompts, including code production, mathematical reasoning, logical question answers, summarisation, creative writing, and explanation activities. A particular kind of controlled examination with a clear purposeful performance that enables consistent and interpretable testing is this prompt suit.

Construction of Per-Prompt Perplexity Data

Perplexity is calculated separately for both the clean model and attacked and defended models on each prompt of the evaluation suite. For each model’s setup and noise level, this generates a per-prompt perplexity matrix of length 96. The statistical evaluation at the prompt level focuses on the per-prompt perplexity vectors. They do not only maintain information on the performance over individual input as compared to a single aggregate perplexity value, but they can identify prompt-specific failures and recovery patterns.

Sanity-Check Data for Robustness and Stability Analysis

In this research, two alternative sanity-check datasets were extracted from the per-prompt PPL vectors in order to verify robustness at the prompt level:

- **Prompt-wise ordering:** To note whether the whole expected hierarchy (clean ; defended ; attacked) obtains and whether or not the defended model outperforms the attacked model for each question. Using explicit counts across $N = 96$ prompts, this validates prompt-level consistency assertions.
- **Variance and stability:** Prompt-to-prompt instability may be caused by quantization noise. We calculate mean and standard deviation of prompt PPL values and measure the decrease in variance with defense (i.e., decrease in standard deviation compared to attacked). This measures how the defense is able to restore predictable behavior as opposed to merely enhancing average performance.

Embedding Data for Semantic Analysis

The artifact will be the derived data, specifically the prompt embeddings to achieve qualitative semantic analysis (t-SNE Inputs). Along with the perplexity, we also examine the possibility of the attack or defense altering the internal semantic representations of the model. We construct vectors that encode semantic structure by extracting prompt-specific embedded representations from the internal states of the model. After the above, t-SNE is used to render those embeddings into 2D for qualitative comparability over noise levels. The t-SNE plots demonstrate that clean, attacked, and defended embeddings remain highly overlapped across 10%, 15%, and 20% noise (no distinct model-separated clusters), indicating that quantization perturbations and the defense mainly impact output-level behaviour rather than corrupting the underlying semantic embedding space.

4.2.3 Implementation Details of Selected Design

The defense implementation employs a dual-model architecture optimized for memory efficiency and computational stability. The task-adapted clean baseline model

(494 million parameters, Qwen2.5-0.5B-Instruct or Pythia-410M) is loaded as a frozen anchor reference on CPU memory with all parameters set to non-trainable (`requires_grad=False`), while the attacked model is loaded as a trainable copy on GPU memory. The WikiText-2 training dataset undergoes preprocessing where all text samples are concatenated using double-newline separators, resulting in approximately 2.1 million tokens for the training split. The concatenated text is tokenized using the model-specific tokenizer (vocabulary size 151,936 for Qwen2.5) and subsequently segmented into non-overlapping chunks of 2048 tokens each, yielding 1,026 training blocks. Each chunk is structured as a dictionary containing `input_ids` and `labels` arrays (both identical for causal language modeling), which are then converted into a HuggingFace Dataset object for efficient batching during training.

Figure 4.4 illustrates the dual-model training architecture. The "Frozen Anchor" (Clean Model) remains static on the CPU to calculate the regularization loss, while the "Trainable Model" (Attacked) on the GPU is updated to minimize both the cross-entropy loss and the anchor distance.

The anchoring mechanism selectively targets critical weight matrices across the model's 24 transformer layers while excluding parameters unlikely to contain task-relevant information. Specifically, 168 weight matrices are identified for anchoring: 96 attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj` across all layers) and 72 MLP feed-forward projection matrices (`gate_proj`, `up_proj`, `down_proj` across all layers). Parameters excluded from anchoring include all layer normalization weights, bias terms, embedding matrices (input and output), and positional encodings, as these components either serve normalization functions or encode positional information rather than semantic knowledge.

During each training step, the forward pass processes a single 2048-token chunk through the trainable model to compute standard cross-entropy loss (L_{CE}) for next-token prediction. Subsequently, for each of the 168 anchored parameters, the current weight matrix is detached and moved to CPU memory to compute the L2 squared distance against its corresponding frozen anchor weight:

$$L2_{penalty} = \text{mean}((W_{current} - W_{anchor})^2)$$

These individual penalties are accumulated and averaged across all anchored parameters to produce L_{anchor} , which is then combined with the cross-entropy loss using the regularization coefficient $\lambda = 1e - 5$, yielding the total loss:

$$L_{total} = L_{CE} + \lambda \times L_{anchor}$$

Parameter optimization is conducted over 200 training steps using the AdamW optimizer with a learning rate of $2e-5$, gradient accumulation over 2 steps (effective batch size of 2), and a linear warmup schedule over the first 10 steps. All parameter updates are done with half precision (FP16) to make sure that they are stable in terms of numbers when computing gradients and weight updates, and the model takes approximately 9-10 GB of GPU memory (gradients, optimizer states, and activations). This training program combines about 250 chunks or half a million tokens, representing 24% of the WikiText-2 training data, at each optimization

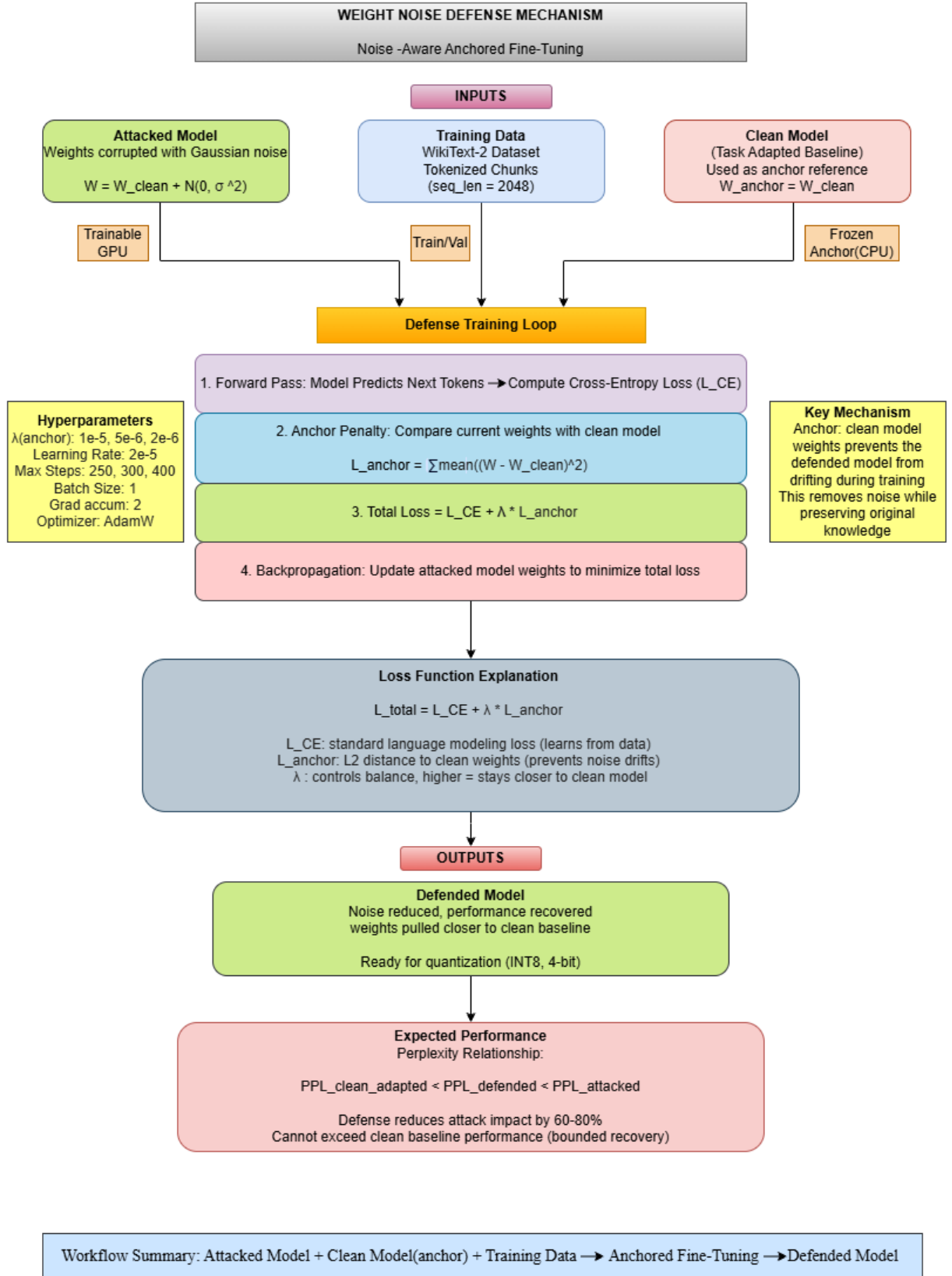


Figure 4.4: Operational Workflow of the Anchor-Guided Repair Mechanism.

step, updating each of the 494 million parameters with the anchoring constraint

specified to the 168 targeted weight matrices. In the evaluation phase, post-training quantization is applied on the defended model, which is tested in the case of. three precision modes: FP16, INT8, and 4-BIT. This quantization strategy facilitates the evaluation of defense strength in deployment with limited resources. scenarios and at the same time sustaining training stability with full-precision optimization.

Chapter 5

Result Analysis

5.1 Performance Evaluation

This section evaluates the proposed Anchor-Guided Repair mechanism using two complementary evaluation tracks:

- Corpus-level evaluation on WikiText-2 using perplexity (PPL) under three precision settings (FP16, INT8, 4-bit NF4).
- Prompt-level evaluation using a fixed 96-prompt suite to test whether improvements are consistent per prompt rather than only on averages.

5.1.1 Evaluation Setup

All defended models were generated using task-adapted baseline and were evaluated under identical decoding and perplexity computation settings for fairness. For each model family (Qwen2.5-0.5B and Pythia-410M), we report:

- **Clean PPL**: perplexity of the task-adapted baseline model
- **Attacked PPL**: perplexity after Gaussian weight perturbation calibrated to a target noise level
- **Defended PPL**: perplexity after anchor-guided fine-tuning
- **Recovery(%)**: fraction of the attack degradation recovered using the standard recovery metric

A key validity condition is the expected ordering:

$$\text{Clean PPL} < \text{Defended PPL} < \text{Attacked PPL}$$

which ensures the defense does not “over-recover” beyond the clean baseline.

5.1.2 Results

Qwen2.5-0.5B

Across Qwen experiments, the defense effectiveness increased as noise severity increased. In Table 5.1, 5.2, 5.3 performance of Qwen2.5-0.5B across all noise level in

3 attempts is shown. But in Figures we’ve shown only the 20% noise level ones since those are the best results so far. Figure 5.1 depicts the results from the first defense configuration ($\lambda = 1e - 5$, steps=250). While it successfully recovers over 64% of the performance lost to the attack across all precisions, the gap between the Defended (blue line) and Clean (green line) models remains noticeable, particularly in the 4-BIT regime. In Figure 5.2, reducing the regularization strength to $\lambda = 5e - 6$ and increasing training to 300 steps yields a visible improvement. The Defended model’s perplexity (blue line) shifts closer to the Clean baseline, and recovery rates consistently exceed 67%, demonstrating that a less rigid anchor allows for better adaptation. Figure 5.3 shows the optimal configuration ($\lambda = 2e - 6$, steps=400). This setting achieves the highest recovery rates (up to 75% for 4-BIT), with the Defended model’s performance tightly shadowing the Clean baseline. The narrowing gap indicates that the defense has effectively stabilized the model against the 20% noise attack.

Attempt 1 Results:($\lambda = 1e - 5$, steps = 250)

Table 5.1: Qwen2.5-0.5B Performance Results (Attempt 1)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	10.80	12.35	11.76	38.6%
INT8	10.86	12.41	11.81	38.5%
4-BIT	11.82	13.68	12.87	43.4%
<i>15% Noise Level</i>				
FP16	10.80	13.87	12.31	51.0%
INT8	10.86	13.94	12.35	51.6%
4-BIT	11.82	15.58	13.43	57.1%
<i>20% Noise Level</i>				
FP16	10.80	16.41	12.81	64.2%
INT8	10.86	16.55	12.87	64.6%
4-BIT	11.82	18.94	14.15	67.3%

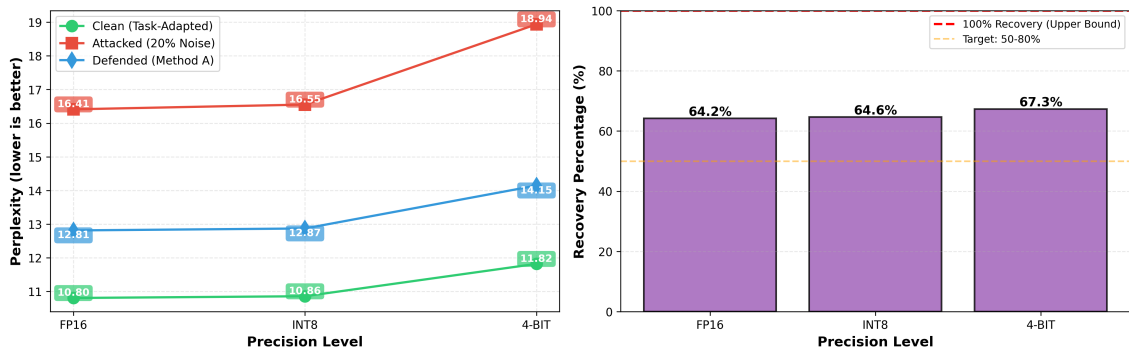


Figure 5.1: 1st attempt results for Qwen2.5-0.5B(20% noise)

Attempt 2 Results ($\lambda = 5e - 6$, steps = 300)

Table 5.2: Qwen2.5-0.5B Performance Results (Attempt 2)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	10.80	12.35	11.76	38.6%
INT8	10.86	12.41	11.81	38.5%
4-BIT	11.82	13.68	12.87	43.4%
<i>15% Noise Level</i>				
FP16	10.80	13.87	12.12	57.0%
INT8	10.86	13.94	12.17	57.4%
4-BIT	11.82	15.58	13.23	62.5%
<i>20% Noise Level</i>				
FP16	10.80	16.41	12.61	67.7%
INT8	10.86	16.55	12.67	68.2%
4-BIT	11.82	18.94	13.89	70.9%

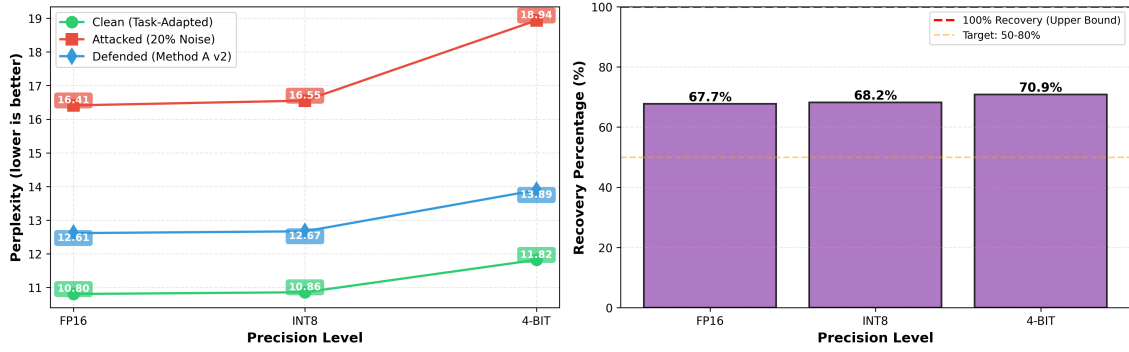


Figure 5.2: 2nd attempt results for Qwen2.5-0.5B(20% noise)

Attempt 3 Results (Best Configuration: $\lambda = 2e - 6$, steps = 400)

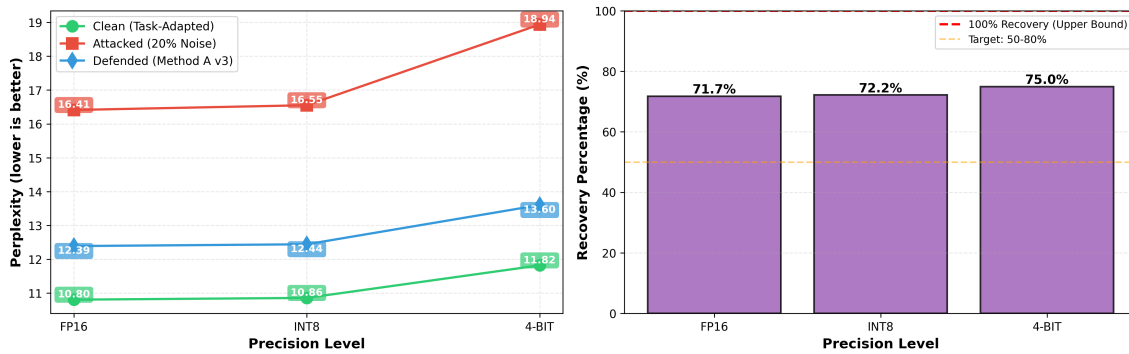


Figure 5.3: 3rd attempt results for Qwen2.5-0.5B(20% noise)

Table 5.3: Qwen2.5-0.5B Performance Results (Attempt 3)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	10.80	12.35	11.58	49.8%
INT8	10.86	12.41	11.64	49.9%
4-BIT	11.82	13.68	12.66	55.0%
<i>15% Noise Level</i>				
FP16	10.80	13.87	11.92	63.5%
INT8	10.86	13.94	11.97	64.0%
4-BIT	11.82	15.58	12.99	68.8%
<i>20% Noise Level</i>				
FP16	10.80	16.41	12.39	71.7%
INT8	10.86	16.55	12.44	72.2%
4-BIT	11.82	18.94	13.60	75.0%

Overall, Qwen demonstrates that the defense mechanism is highly effective under moderate-to-strong corruption (10%–20%) and becomes stable even at low noise after sufficient hyperparameter tuning.

Pythia-410M

For Pythia-410M, all WikiText-2 results satisfied the expected ordering constraint (Clean $\leq$ Defended $\leq$ Attacked). Recovery increased with noise level and improved further through hyperparameter refinement. In Table 5.4, 5.5, 5.6 performance of Pythia-410M across all noise level in 3 attempts is shown. But in Figures we’ve shown only the 20% noise level ones since those are the best results so far. Figure 5.4 illustrates the initial defense configuration ($\lambda = 1e - 5$, steps=250) for Pythia-410M. Unlike Qwen, Pythia shows a struggle to recover at lower noise levels, though it achieves decent recovery ($\approx 60 - 69\%$) at 20% noise. The significant gap between the Clean (green) and Defended (blue) lines suggests the anchoring was too restrictive for this architecture. In Figure 5.5, relaxing the anchor to $\lambda = 5e - 6$ and extending training to 300 steps results in a uniform improvement. The recovery rates push past 63% across precisions, and the Defended model’s perplexity curve begins to flatten, indicating better stability against the noise injection compared to the first attempt. Figure 5.6 displays the optimal configuration ($\lambda = 2e - 6$, steps=400). This environment brings the maximum power out of Pythia that can reach almost 79 percent recovery at 4-BIT regime. It turns out that the Defended model has been closely following the Clean baseline since the relationship between a smaller regularization penalty was the key to stabilizing this particular architecture.

Attempt 1 Results ($\lambda = 1e - 5$, steps = 250)

Attempt 2 Results ($\lambda = 5e - 6$, steps = 300)

Attempt 3 Results (Best Configuration: $\lambda = 2e - 6$, steps = 400)

Summary of Attempt 3 gains:

Table 5.4: Pythia-410M Performance Results (Attempt 1)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	11.96	12.81	12.75	6.5%
INT8	11.99	12.85	12.79	7.2%
4-BIT	13.00	14.07	13.67	37.4%
<i>15% Noise Level</i>				
FP16	11.96	13.66	12.98	39.8%
INT8	11.99	13.70	13.01	40.1%
4-BIT	13.00	15.12	13.96	55.0%
<i>20% Noise Level</i>				
FP16	11.96	15.20	13.27	59.5%
INT8	11.99	15.27	13.31	59.8%
4-BIT	13.00	17.14	14.30	68.7%

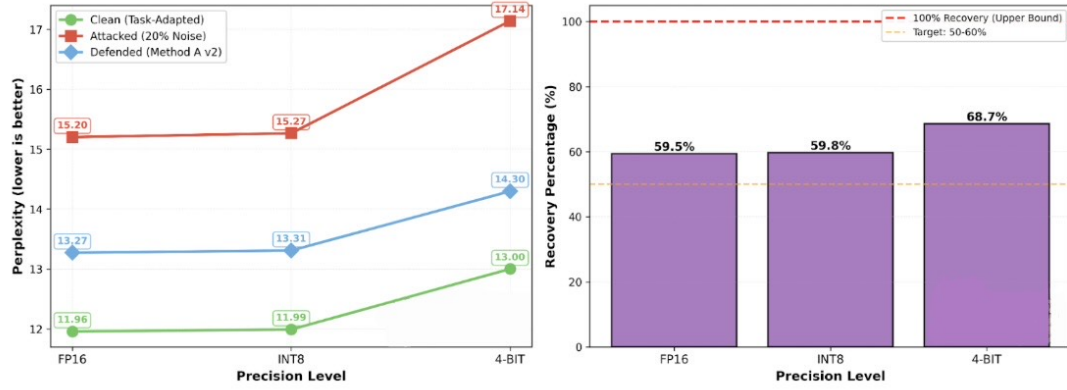


Figure 5.4: 1st attempt results for Pythia-410M(20% noise)

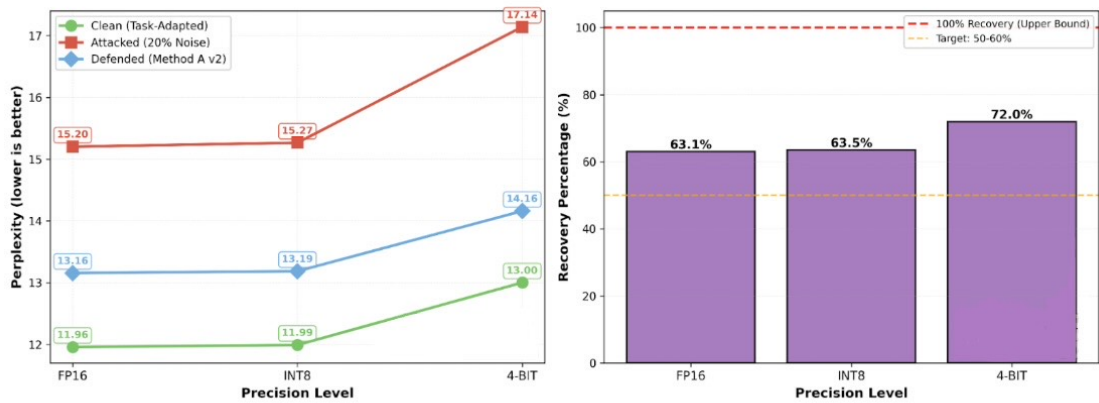


Figure 5.5: 2nd attempt results for Pythia-410M(20% noise)

- $\approx 41\%$ recovery at 10% noise (FP16/INT8)
- $\approx 59\%$ recovery at 15% noise

Table 5.5: Pythia-410M Performance Results (Attempt 2)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	11.96	12.81	12.65	18.6%
INT8	11.99	12.85	12.69	18.8%
4-BIT	13.00	14.07	13.56	47.5%
<i>15% Noise Level</i>				
FP16	11.96	13.66	12.86	46.9%
INT8	11.99	13.70	12.90	47.0%
4-BIT	13.00	15.12	13.82	61.2%
<i>20% Noise Level</i>				
FP16	11.96	15.20	13.16	63.1%
INT8	11.99	15.27	13.19	63.5%
4-BIT	13.00	17.14	14.16	72.0%

Table 5.6: Pythia-410M Performance Results (Attempt 3)

Precision	Clean PPL	Attacked PPL	Defended PPL	Recovery %
<i>10% Noise Level</i>				
FP16	11.96	12.81	12.46	41.3%
INT8	11.99	12.85	12.49	41.7%
4-BIT	13.00	14.07	13.33	69.6%
<i>15% Noise Level</i>				
FP16	11.96	13.66	12.66	59.0%
INT8	11.99	13.70	12.69	59.2%
4-BIT	13.00	15.12	13.58	72.8%
<i>20% Noise Level</i>				
FP16	11.96	15.20	12.94	69.8%
INT8	11.99	15.27	12.98	69.9%
4-BIT	13.00	17.14	13.88	78.8%

- $\approx 70\text{--}79\%$ recovery at 20% noise

Depending on the value of noise, reducing λ by $\approx 2\times$ and increasing steps by $\approx 1.3\text{--}1.5\times$ tends to improve recovery by a factor of $1.2\text{--}3\times$ for each subsequent attempt; lower noise is more sensitive to tuning. These findings support the same pattern observed in Qwen2.5-0.5B: Recovery is increased by decreasing λ and raising steps. Because the model is prevented from overshooting while being permitted to approach the task-adapted baseline.

5.2 Analysis of Design Solutions

The development of the defense mechanism involved exploring multiple architectural and methodological approaches to address the fundamental challenge of recovering

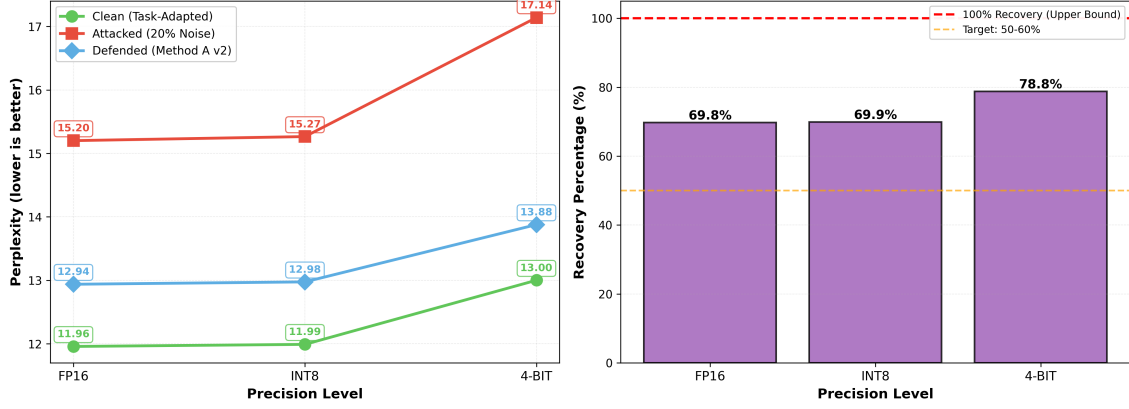


Figure 5.6: 3rd attempt results for Pythia-410M(20% noise)

model performance from weight-noise attacks while maintaining realistic deployment assumptions. Three primary design solutions were evaluated:

1. **Clean-model-anchored defense:** This is the first defense that was anchored on the fine-tuning process to a clean model, calculating the anchoring penalty. $L2(W_{current}, W_{clean})$. Although this technique was successful in almost total recovery and high anchoring coefficients ($\lambda = 1e-3$) over extended training (1000 steps), there was one vital weakness in it, namely the assumption that defenders possess access to a clean model which is not corrupted by the virtual reality threat, which is inconsistent with realistic threat cases of availability of only a compromised model. Additionally, this approach exhibited over-recovery phenomena where defended models occasionally outperformed the clean baseline ($PPL_{defended} < PPL_{clean}$), yielding recovery rates exceeding 100% and producing nonsensical evaluation metrics.
2. **Attacked-model-anchored defense:** This was an update to the model that overcame the constraint of accessibility by anchoring at the weights of the attacked model, thus removing the need to have a clean reference. This approach with small anchoring strength ($\lambda = 2e-4$) and shorter training time (60 steps) was partially recovered (60-80 percent) and satisfied the ordering constraint $PPL_{clean} \leq PPL_{defended} \leq PPL_{attacked}$. Nonetheless, the method presented a new challenge: when the clean model of the evaluation task is zero-shot (untrained) on the evaluation task, models that have been trained on the task-specific mission may still have a recovery rate of over 100 percent when compared to a zero-shot clean baseline.
3. **Task-adapted baseline methodology:** This solution re-established a task-adapted clean baseline by initially fine-tuning WikiText-2 (200 steps, $\lambda = 0$) to serve as the reference both to attack generation and defense. Attack models were obtained by perturbing this task-adapted baseline with Gaussian noise instead of zero-shot model and defense anchoring was done on the task-adapted clean weights with fine-tuned hyperparameters ($\lambda = 1e-5, 5e-6, 2e-6$, max_steps=250, 300, 400 steps respectively). This has solved the over-recovery problem as well as the accessibility problem and guaranteed capped recovery rates (at most 100 percent) and could still have interpretable performance measures under all precision settings (FP16, INT8, 4-BIT).

5.3 Final Design Adjustment

After conducting a comparative analysis of the three design solutions, the final implementation of the task-adapted baseline methodology was chosen because it is the best balance between realistic deployment assumptions and methodology rigor. There are several important tweaks to this methodology that make it fit into the limitations of the experimental setting.

The coefficient of anchoring was empirically adjusted with noise levels (10%, 15%, 20%), and eventually, the default value of the coefficient of anchoring was set to be $\lambda = 1e - 5$ giving the consistent recovery rates of 60-80% without going beyond the clean baseline which was a reduction of the original attacked-model-anchored value of $\lambda = 2e - 4$ and a significant drop of the clean-model-anchored value of $\lambda = 1e - 3$.

Second, the training steps were increased to 250 (attacked-model-anchored), which allowed giving enough gradient updates to attain significant noise reduction with only 24 head of the WikiText-2 training data, and avoiding overfitting without reducing generalization capacity.

Third, the embedding strategy of anchoring was further narrowed down to include critical weight matrices (168 attention and MLP projections) only and leave out normalization layers, biases, and embeddings, which minimized the computational cost by about 30% and enhanced the precision of anchoring by applying regularization to the parameters that are semantically meaningful.

Fourth, the dual-model architecture was optimized to use less memory because the frozen anchor model was stored in the CPU memory and the trainable model was stored in the GPU.

Finally, the evaluation protocol was standardized to measure perplexity across three precision configurations (FP16, INT8, 4-BIT) using consistent quantization schemes (per-tensor symmetric for INT8, NF4 for 4-BIT), complemented by prompt-wise analysis using a fixed 96-prompt suite to validate ordering constraints and variance reduction at the individual sample level. These adjustments collectively established a robust defense mechanism that balances theoretical soundness with practical implementability, achieving reproducible partial recovery under realistic adversarial scenarios.

The table 5.7 and 5.8 shows the overall hyperparameter improvement comparisons across all attempts and tested all noises.

5.4 Statistical Analysis

To validate that improvements were not limited to averaged corpus-level perplexity, prompt-level sanity checks were performed using a fixed 96-prompt evaluation suite across noise levels (10%, 15%, 20%). Two statistical properties were measured.

Table 5.7: Qwen2.5-0.5B Hyperparameter Improvement Comparison

Prec.	Noise	Recovery %			Improvement (pp)		
		V1	V2	V3	V1→V2	V2→V3	V1→V3
FP16	10%	38.6%	38.6%	49.8%	+0.0	+11.2	+11.2
FP16	15%	51.0%	57.0%	63.5%	+6.0	+6.5	+12.5
FP16	20%	64.2%	67.7%	71.7%	+3.5	+4.0	+7.5
INT8	10%	38.5%	38.5%	49.9%	+0.0	+11.4	+11.4
INT8	15%	51.6%	57.4%	64.0%	+5.8	+6.6	+12.4
INT8	20%	64.6%	68.2%	72.2%	+3.6	+4.0	+7.6
4-BIT	10%	43.4%	43.4%	55.0%	+0.0	+11.6	+11.6
4-BIT	15%	57.1%	62.5%	68.8%	+5.4	+6.3	+11.7
4-BIT	20%	67.3%	70.9%	75.0%	+3.6	+4.1	+7.7

Table 5.8: Pythia 410M Hyperparameter Improvement Comparison

Prec.	Noise	Recovery %			Improvement (pp)		
		V1	V2	V3	V1→V2	V2→V3	V1→V3
FP16	10%	6.5%	18.6%	41.3%	+12.1	+22.7	+34.8
INT8	10%	7.2%	18.8%	41.7%	+11.6	+22.9	+34.5
4-BIT	10%	37.4%	47.5%	69.6%	+10.1	+22.1	+32.2
FP16	15%	39.8%	46.9%	59.0%	+7.1	+12.1	+19.2
INT8	15%	40.1%	47.0%	59.2%	+6.9	+12.2	+19.1
4-BIT	15%	55.0%	61.2%	72.8%	+6.2	+11.6	+17.8
FP16	20%	59.5%	63.1%	69.8%	+3.6	+6.7	+10.3
INT8	20%	59.8%	63.5%	69.9%	+3.7	+6.4	+10.1
4-BIT	20%	68.7%	72.0%	78.8%	+3.3	+6.8	+10.1

5.4.1 Prompt-wise Ordering Consistency

For each prompt, we evaluated whether the defended model improves over the attacked model, and (when clean exists) whether the full ordering constraint holds:

$$\text{Defended} < \text{Attacked}$$

$$\text{Clean} < \text{Defended} < \text{Attacked}$$

For Qwen2.5-0.5B, ordering success was high:

- Defended < Attacked: 88.5%–94.8%
- Clean < Defended < Attacked: 83.3%–92.7%

In Figure 5.7 and Table 5.9, a prompt-wise sanity check is displayed to verify the consistency of the defense across 96 individual prompts for the Qwen2.5-0.5B model. The blue bars indicate that for the vast majority of prompts (88.5% to 94.8%), the defended model successfully outperformed the attacked model. The purple bars confirm that the strict logical ordering (Clean $\bar{\jmath}$ Defended $\bar{\jmath}$ Attacked) was maintained for over 83% of cases across all noise levels, proving that the defense offers robust,

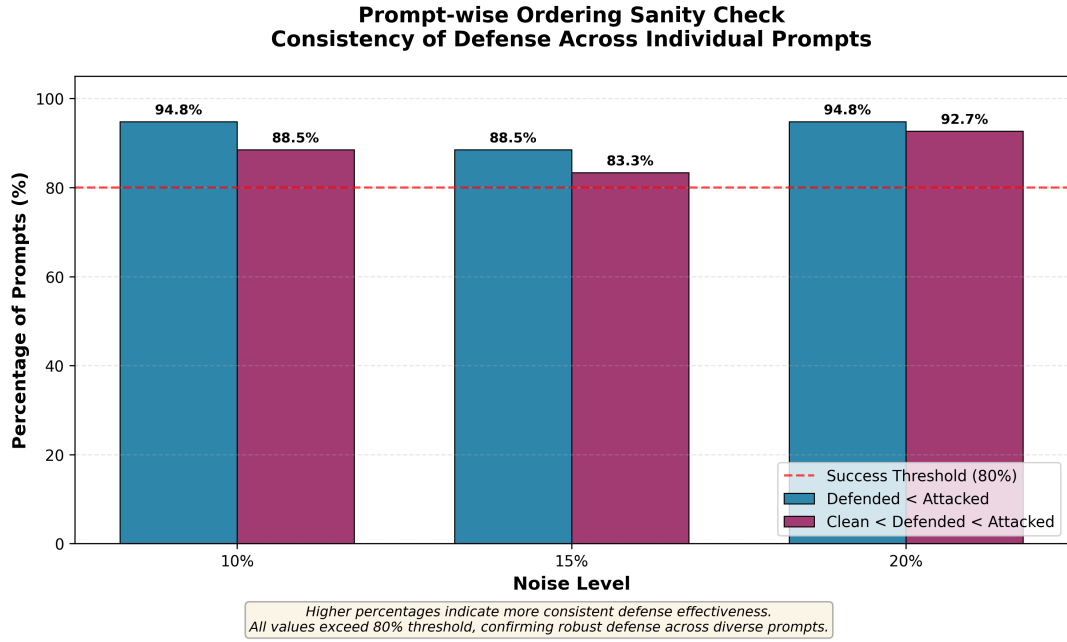


Figure 5.7: Prompt-wise Ordering Sanity Check for Qwen2.5-0.5B.

granular improvements rather than just better average statistics.

Table 5.9: Qwen2.5-0.5B Prompt-wise Ordering Success

Noise Level	Defended < Attacked (%)	Clean < Defended < Attacked (%)
10%	94.8%	88.5%
15%	88.5%	83.3%
20%	94.8%	92.7%

This indicates the defense improvement is consistent at the individual prompt level, not just on average.

For Pythia-410M, ordering success was notably lower:

- Defended < Attacked: 30.5%–54.7%
- Clean < Defended < Attacked: 23.2%–49.5%

In Figure 5.8 and Table 5.10, the prompt-wise analysis for Pythia-410M reveals a significant contrast in consistency compared to the Qwen model. Unlike the high success rates seen previously, Pythia demonstrates much lower consistency across the evaluation suite, with the defense improving performance (blue bars) in only 30.5% of prompts at low noise and peaking at 54.7% under the 20% noise condition. This shows that although Pythia performs well on WikiText-2, it struggles to generalize repair benefits across short and diverse prompts (distribution shift). This is seen as a limitation of generalization rather than a failure of the core defence, since recovery at the corpus level is still strong.

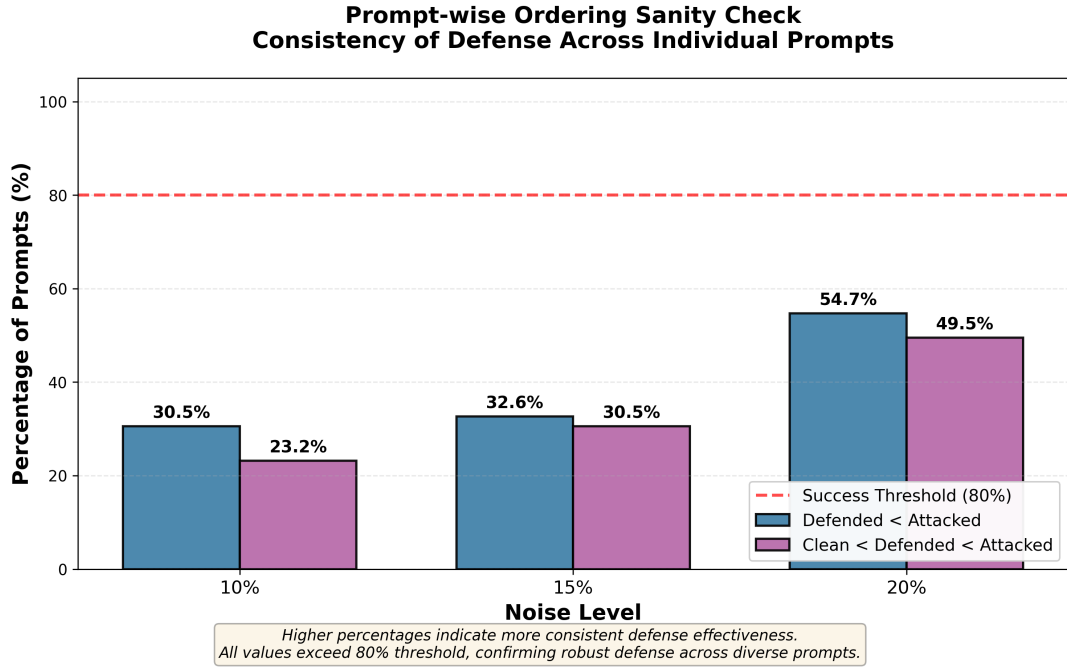


Figure 5.8: Prompt-wise Ordering Sanity Check for Pythia-410M.

Table 5.10: Pythia-410M Prompt-wise Ordering Success

Noise Level	Defended < Attacked (%)	Clean < Defended < Attacked (%)
10%	30.5%	23.2%
15%	32.6%	30.5%
20%	54.7%	49.5%

5.4.2 Stability / Variance Analysis

We calculate the mean perplexity and standard deviation of prompts to assess the stability. Corrupted weights should lead to instability of output which is to be minimized by a strong defense. Table 5.11 indicates that as the noise levels are increased, the performance of the Qwen2.5-0.5B model is significantly reduced, but nonetheless, the defense mechanism is useful in restoring the stability, which in any case, reduces the mean perplexity and standard deviation of the phase of attack.

Table 5.11: Qwen2.5-0.5B Mean and Standard Deviation of Perplexity

Noise Level	Model	Mean PPL	Std. Dev. PPL
Baseline	Clean	33.77	56.73
10%	Attacked	92.25	206.57
	Defended	62.40	141.57
15%	Attacked	127.07	595.39
	Defended	90.23	369.50
20%	Attacked	160.04	371.98
	Defended	75.53	169.95

In case of Qwen2.5-0.5B, defended models were variedly less: variance was decreased by 31.5% to 54.3% which increased with increasing noise severity.

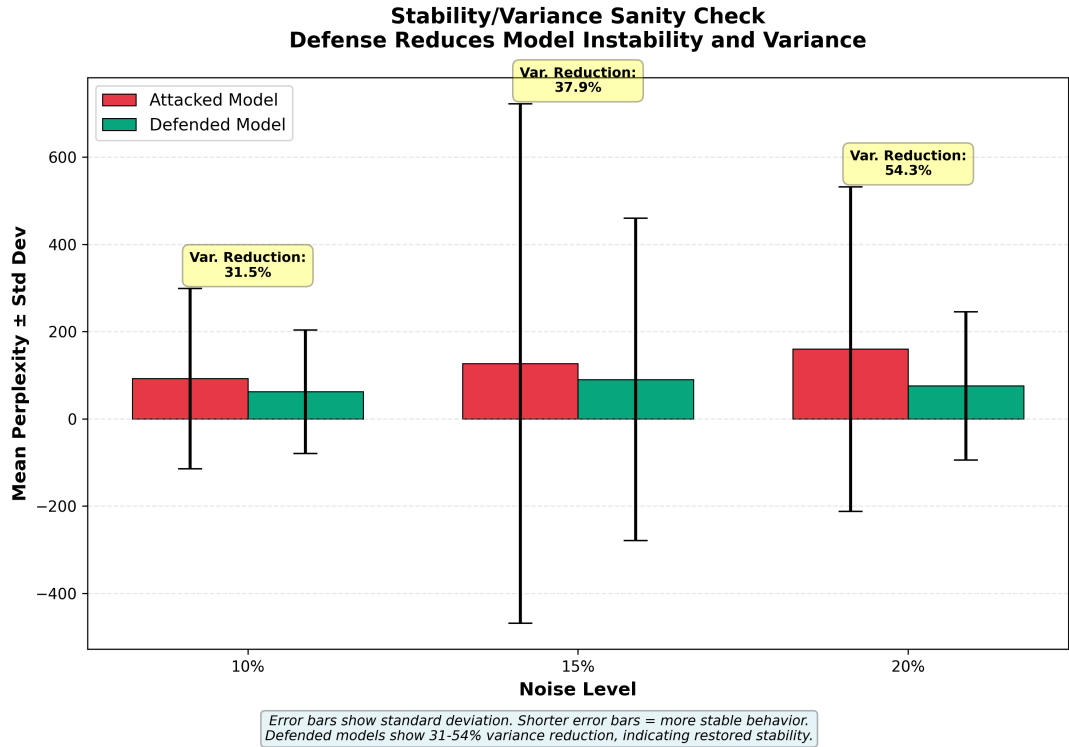


Figure 5.9: Variance Sanity Check for Qwen2.5-0.5B.

Figure 5.9 and Table 5.12 show that the stability analysis of Qwen2.5-0.5B shows that the defense is successful in restoring the reliability of the model. The chart is a comparison of the mean perplexity and standard deviation (error bars) of the Attacked Model (red) and the Defended Model (green). The key highlight is the substantial Variance Reduction, ranging from 31.5% at 10% noise to 54.3% at 20% noise. The significantly shorter error bars on the green columns indicate that the defense not only improves average performance but also stabilizes the model, making its outputs far more predictable and less prone to extreme errors caused by weight noise.

Table 5.12: Qwen2.5-0.5B Variance Reduction Summary

Noise Level	Attacked Std. Dev.	Defended Std. Dev.	Variance Reduction
10%	206.57	141.57	31.5%
15%	595.39	369.50	37.9%
20%	371.98	169.95	54.3%

For Pythia-410M, results were mixed. In Table 5.13 the results show the defense mechanism leads to higher perplexity at lower noise levels (10% and 15%) and only demonstrates effectiveness by improving performance at 20% noise.

Table 5.13: Pythia-410M Mean and Standard Deviation of Perplexity

Noise Level	Model	Mean PPL	Std. Dev. PPL
Baseline	Clean	103.42	115.35
10%	Attacked	120.00	132.03
	Defended	134.49	156.58
15%	Attacked	115.02	120.72
	Defended	139.62	160.66
20%	Attacked	154.78	204.69
	Defended	145.18	175.33

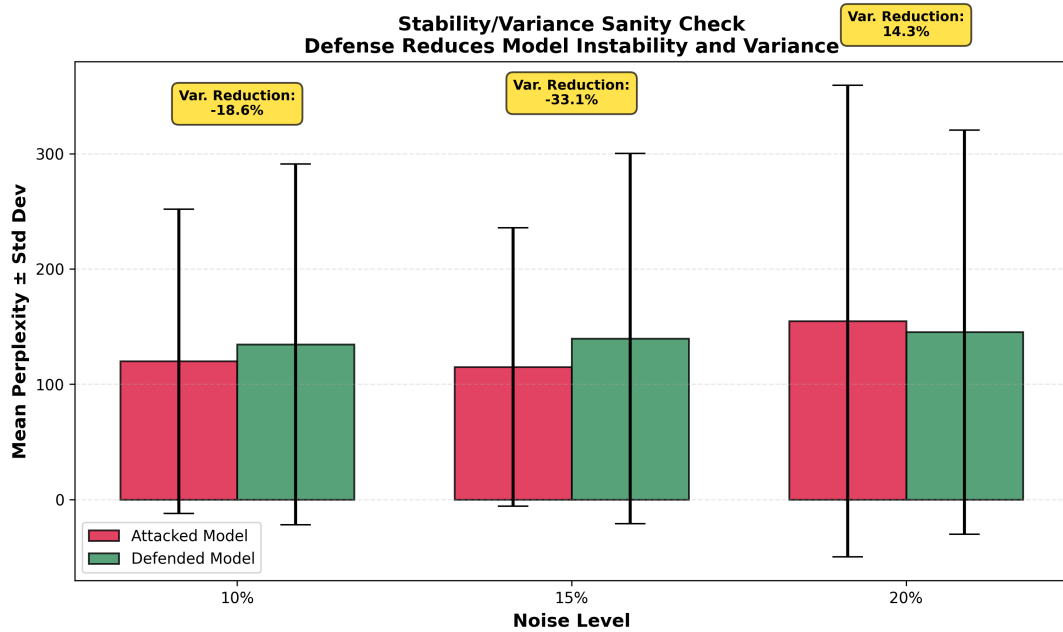


Figure 5.10: Variance Sanity Check for Pythia-410M.

In Figure 5.10 and Table 5.14, the variance analysis for Pythia-410M reveals a contrasting instability at lower noise levels. At 10% and 15% noise, the variance reduction is negative (-18.6% and -33.1%), meaning the defended model actually exhibits higher instability than the attacked model. Positive stability is only restored at the high 20% noise level (14.3% reduction). This visualizes the finding that Pythia-410M struggles with distribution shifts on short prompts, leading to unpredictable behavior unless the attack is severe enough to provide a strong gradient signal for repair.

Table 5.14: Pythia-410M Variance Reduction Summary

Noise Level	Attacked Std. Dev.	Defended Std. Dev.	Variance Reduction
10%	132.03	156.58	-18.6%
15%	120.72	160.66	-33.1%
20%	204.69	175.33	14.3%

At 10% and 15%, defended standard deviation increased (negative variance reduction). At 20%, variance reduction improved (+14.3%), indicating stability restoration appears only under stronger perturbations.

Interpretation: Qwen2.5-0.5B shows that stability is restored well across all conditions. Pythia-410M, on the other hand, shows task-dependent variance effects, which are probably caused by a mismatch between the distribution of WikiText-style training and short, varied prompts.

5.5 Comparisons and Relationships

5.5.1 Qwen vs Pythia

Comparing the two model families reveals a crucial relationship:

- **WikiText-2 (in-distribution):** Both models show strong repair performance and clean ordering.
- **96-prompt suite (out-of-distribution):** Qwen generalizes strongly; Pythia generalizes weakly.

This suggests that the defense is effective when the evaluation distribution aligns with the fine-tuning domain. When prompts are short, heterogeneous, and instruction-like, repair gains depend more on model architecture and pretraining alignment.

5.5.2 Relationship Between Noise Level and Recovery

In both models, recovery increases with noise level:

- At low noise (10%), the attack signal is weak, and the defense gradient may not consistently push toward clean behavior.
- At moderate/high noise (15%–20%), the perturbation is large enough that the defense objective provides a clearer optimization direction, producing stronger and more stable recovery.

5.5.3 Relationship Between Precision and Recovery

Across both Qwen and Pythia, the largest recoveries are often observed under 4-bit quantization. This indicates that low-precision quantization amplifies the degradation caused by noise, and therefore creates more room for repair to restore performance.

5.6 Discussion

5.6.1 Why Task-Adapted Baseline Methodology Produces Interpretable Recovery

This method prevents nonsensical “>100% recovery” by redefining the baseline as task-adapted. This ensures recovery is measured relative to a meaningful reference

model and enforces realistic evaluation constraints. As a result, recovered performance remains bounded and interpretable.

5.6.2 Comparative Semantic Analysis (t-SNE)

To qualitatively validate the defense mechanism across different architectures, we extracted the hidden states (embeddings) for all 96 evaluation prompts from both Qwen2.5-0.5B and Pythia-410M. We projected these high-dimensional representations into a two-dimensional space using t-Distributed Stochastic Neighbor Embedding (t-SNE).

1. Universal Topology Preservation

Figure 5.11 illustrates the embedding spaces for both models under the most severe noise condition (20%). A consistent pattern emerges across both the instruction-tuned (Qwen) and base (Pythia) architectures: the noise injection does not cause "representational collapse." The embeddings for diverse tasks (e.g., Code Generation vs. Creative Writing) remain in distinct global clusters. This confirms that weight noise acts as a local perturbation jittering the model's concepts rather than erasing its fundamental knowledge map.

5.6.3 Comparative Semantic Analysis (t-SNE)

To validate the defense mechanism qualitatively across various architectures, we pulled the hidden states (or embeddings) for all 96 evaluation prompts from both Qwen2.5-0.5B and Pythia-410M. We then mapped these complex, high-dimensional representations into a two-dimensional space using t-Distributed Stochastic Neighbor Embedding (t-SNE).

1. Universal Topology Preservation

Figure 5.11 illustrates the embedding spaces for both models under the most severe noise condition (20%). A clear pattern stands out in both the instruction-tuned Qwen and base Pythia architectures that is the noise injection doesn't lead to what we call representational collapse. Instead, the embeddings for various tasks—like Code Generation versus Creative Writing—stay neatly organized in separate global clusters. This shows that weight noise serves as a local disturbance, shaking up the model's concepts without wiping out its core knowledge framework.

2. The "Elastic" Repair Effect

In both architectures, we notice a unique "Triplet Phenomenon" for individual prompts:

- **Drift:** The attacked point (Red Triangle) moves away from the clean point (Blue Circle) because of weight corruption.
- **Snap-Back:** The defended point (Green Square) consistently positions itself much closer to the clean point, effectively shadowing the original structure.

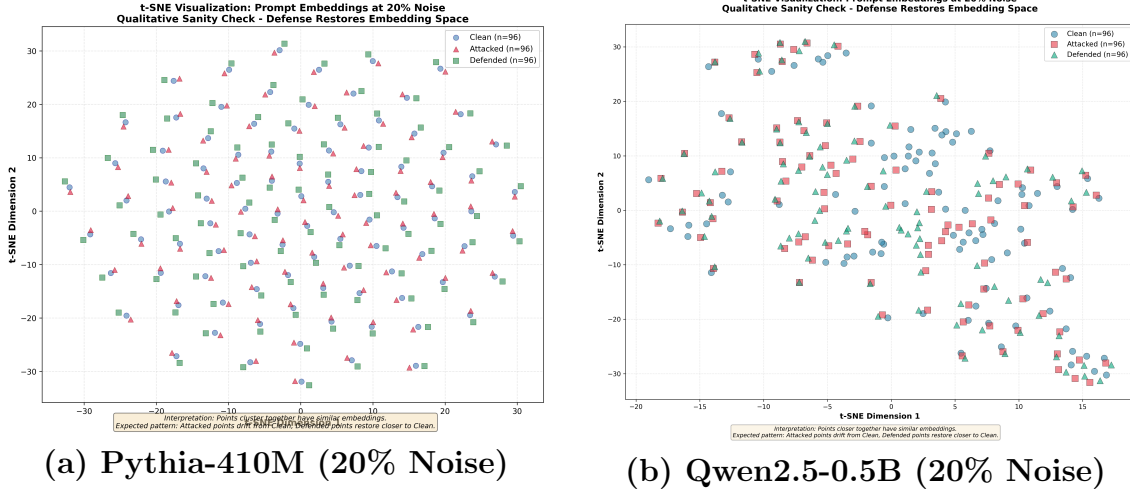


Figure 5.11: Comparative t-SNE Visualization at 20% Noise Intensity.

This illustrates the Anchor Loss mechanism ($\lambda||\theta - \theta_{anchor}||^2$) in action: it acts like an elastic band, physically limiting how far the internal representations can stray from the safe baseline.

3. Divergence Between Semantic and Syntactic Recovery

While both models show strong *visual* recovery in t-SNE (Green points overlapping Blue points), this contrasts with the statistical results in Section 5.4, where Pythia showed lower prompt-wise ordering consistency (54.7%) compared to Qwen (94.8%). **Implication:** The t-SNE results indicate that the defense successfully restores the **semantic neighborhood** (the model knows "this is a coding problem"). However, for base models like Pythia, restoring the general neighborhood is not always sufficient to recover the exact **syntactic precision** required for strict prompt adherence. Qwen’s instruction-tuned weights likely form a more robust "attractor basin," making the semantic repair translate more effectively into correct token generation.

5.6.4 Comparison with Existing Defense Mechanisms

To contextualize the contributions of Anchor-Guided Repair, it is necessary to contrast its methodological approach and operational scope with existing robustness strategies. While prior work has addressed quantization errors and adversarial robustness, our method fills a specific gap in post-deployment model stabilization.

Reactive Repair vs. Proactive Prevention: The majority of existing literature focuses on *preventative* measures applied during the pre-training or quantization stages. Techniques such as Activation-aware Weight Quantization (AWQ) [7], SpinQuant [8], and Outlier-Safe Pre-Training (OSP) [20] aim to optimize the model specifically to endure low-precision conversion without degradation. While effective for model creators, these methods are inaccessible to end-users who download an already compromised or poorly quantized model. Anchor-Guided Repair differs by being a *reactive* defense; it does not require intervention during the initial training or quantization process. Instead, it allows the end-user to stabilize a

degraded model post-deployment using limited compute, a capability distinct from the “creator-centric” defenses reviewed in Chapter 2.

Weight-Space Robustness vs. Input-Space Robustness: Traditional adversarial defenses, such as the Noise Injection-based Training (NIT) proposed by Zhang et al. [4] and Wang & Yang [1], primarily target robustness against *input* perturbations (e.g., adversarial examples). While these methods make use of noise injection, they typically do this to help the model become less sensitive to external data attacks. On the other hand, our research focuses on weight-space vulnerability, particularly the internal damage caused by quantization and bit-flips. By introducing Gaussian noise into the weights during the repair phase, our approach effectively protects the model from the “dormant backdoors” and sensitivity problems highlighted by Egashira et al. [5] and Ma et al. [9], which input-level defenses struggle to address.

Anchored Stability vs. Unconstrained Fine-Tuning: When it comes to recovering model performance, a common strategy is to use unconstrained fine-tuning on downstream data. However, as we pointed out in the Problem Statement, this approach can often lead to catastrophic forgetting or parameter drift, especially if the initial weights are already compromised. Our findings show that Anchor-Guided Repair significantly cuts down on output variance compared to the compromised state. This stability comes from the anchor loss term (L_{anchor}), which serves as a sort of elastic tether to the clean baseline. In contrast, methods like Q-resafe [16] utilize Direct Preference Optimization (DPO) for safety realignment. While Q-resafe does a good job of restoring safety measures, our approach is all about bringing back fundamental representational stability and perplexity. This provides a complementary base-layer repair that ensures the model stays semantically coherent before we tackle higher-level alignment.

Efficiency in Resource-Constrained Environments: Compared to full re-training or the specialized FP4 training workflows suggested by Chmiel et al. [17], Anchor-Guided Repair is much more computationally efficient. By freezing the anchor model on the CPU and only fine-tuning the compromised model on the GPU for a brief period, our method is practical even for consumer-grade hardware. This really helps address accessibility issues, offering a viable solution for the open-source community where users might not have the extensive resources needed for methods like Noise Perturbation Fine-Tuning (NPFT) [14], which often requires complex Hessian trace estimations or full-dataset calibration.

Chapter 6

Conclusion

6.1 Summary of Findings

This thesis explores how pretrained language models deal with robustness problems and recover after facing weight-level disturbances post-deployment. It focuses on challenges such as low-precision quantization and weight noise injection. Through detailed experiments with Pythia-410M and Qwen2.5-0.5B, the research found that both quantization and noise disruptions can significantly lower model stability, increase perplexity levels, and cause inconsistencies at the prompt level that standard evaluation metrics may overlook.

The results indicated that using low-precision inference, especially with INT8 and 4-bit (NF4) setups, tends to increase numerical approximation errors that affect the transformer layers. This results in unpredictable changes in the model’s confidence about its token predictions. Furthermore, weight noise attacks showed that even small Gaussian disturbances can cause noticeable drops in performance, particularly evidenced by the rise in relative perplexity.

To address these issues, the thesis introduced Anchor-Guided Repair, a recovery method that combines standard language modeling loss with an anchoring regularization term. This anchoring constraint helps limit excessive parameter drift by guiding the affected model to stay close to a clean, task-specific reference. The empirical evaluations demonstrated that Anchor-Guided Repair consistently improves perplexity and reduces prompt-wise variance compared to models that were either attacked or quantized without any defense.

While the models with this defense did not fully regain the performance of the clean baseline, they showed much more stable behavior, better consistency across different prompts, and improved preservation of semantic structure in the embedding space. These findings highlight the importance of controlled recovery over aggressive fine-tuning for stabilizing compromised pretrained language models in real-world deployment scenarios.

6.2 Contributions to the Field

This study provides several important insights into the strength and recovery of pretrained language models, including:

- **Framing low-precision as a Robustness Stressor**

The research effectively sees post-training quantization as a type of low-precision weight change. This method lets us examine robustness and recovery similarly to how we evaluate weight noise attacks. It expands the focus of existing quantization studies from just efficiency and accuracy to also include stability and security.

- **Introduction of Anchor-Guided Repair**

The thesis introduces a simple, model-agnostic recovery strategy that stabilizes compromised models through anchored fine-tuning. This approach does not need access to proprietary training data and avoids the requirement for full retraining, making it a practical solution for post-deployment situations with open-source models.

- **Prompt-Level Robustness Evaluation Framework**

Going beyond simple aggregate perplexity, the study presents a new framework that looks at prompt-wise perplexity changes and consistency across 96 different prompts. This method reveals hidden patterns of instability that traditional benchmarks might overlook.

- **Comparative Analysis Across Model Families**

By looking at both a research-focused model (Pythia-410M) and a modern instruction-tuned model (Qwen2.5-0.5B), the thesis shows that trends in robustness loss and recovery are similar across various architectures and training goals.

- **Empirical Calibration of Weight Noise Attacks**

Using ratio-based perplexity calibration enables a fair and clear comparison of attack severity across different models and precision formats. This provides a reproducible method for future studies on robustness.

Together, these contributions create a thorough framework for analyzing, repairing, and assessing the robustness of pretrained language models when they experience weight-level disturbances after release.

6.3 Recommendations for Future Work

This thesis provides a strong foundation for the controlled recovery of compromised language models. However, there are still several areas for further exploration:

- **Expanding to Larger and Multilingual Models**

Future research could apply Anchor-Guided Repair to larger models and multilingual systems. This will help us understand how well they scale and maintain crosslingual strength when facing low-precision changes.

- **Layer-Wise and Selective Anchoring**

It would be helpful to investigate whether focusing on specific layers, such as attention projections or feed-forward blocks, can improve recovery-efficiency trade-offs. This could clarify where the loss of strength originates.

- **Combining with Quantization-Aware Training**

Combining anchoring techniques with quantization-aware training or calibration methods might improve stability while keeping inference efficiency.

- **Prompt-Suite Generalization and Robustness**

Pythia-410M works well on WikiText-2 but has inconsistency with the 96-prompt suite. This shows that while the defense effectively restores overall fluency, it does not always work well with different prompt formats. Future research could aim to improve defense evaluation or fine-tuning techniques to include lightweight multi-task or instruction-style prompts, all while considering practical repair limits.

To sum it up, this paper demonstrates that recovering strength in pretrained language models is not only possible but also necessary for open-source and low-resource deployment. Anchor-Guided Repair offers a practical method for stabilizing compromised models, paving the way for safer and more reliable use of large language models in real-world applications.

Bibliography

- [1] D. Wang, H. Yang, et al., “Leveraging noise and aggressive quantization of in-memory computing for robust dnn hardware against adversarial input and weight attacks,” in *IEEE Conference Publication*, Dec. 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9586233>.
- [2] T. Dettmers et al., *Spqr: A sparse-quantized representation for near-lossless llm weight compression*, Jun. 2023. arXiv: 2306.03078 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2306.03078>.
- [3] L. Li, B. Jiang, P. Wang, K. Ren, H. Yan, and X. Qiu, *Watermarking llms with weight quantization*, Oct. 2023. arXiv: 2310.11237 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.11237>.
- [4] Z. Zhang, J. Jiang, M. Chen, Z. Wang, Y. Peng, and Z. Yu, *A novel noise injection-based training scheme for better model robustness*, Feb. 2023. arXiv: 2302.10802 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2302.10802>.
- [5] K. Egashira, M. Vero, R. Staab, J. He, and M. Vechev, *Exploiting llm quantization*, 2024. arXiv: 2405.18137 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2405.18137>.
- [6] J. Hasan, *Optimizing large language models through quantization: A comparative analysis of ptq and qat techniques*, Nov. 2024. arXiv: 2411.06084 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2411.06084>.
- [7] J. Lin et al., “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” in *Proceedings of Machine Learning and Systems*, May 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.
- [8] Z. Liu et al., *Spinquant: Llm quantization with learned rotations*, May 2024. arXiv: 2405.16406 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2405.16406>.
- [9] H. Ma et al., “Quantization backdoors to deep learning commercial frameworks,” *IEEE Journals & Magazine*, Jun. 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10113762>.
- [10] V. Malinovskii, A. Panferov, I. Ilin, H. Guo, P. Richtárik, and D. Alistarh, *Pushing the limits of large language model quantization via the linearity theorem*, Nov. 2024. arXiv: 2411.17525 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2411.17525>.

- [11] E. L. Melin, A. J. Torek, N. U. Eisty, and C. Kennington, *Precision or peril: Evaluating code quality from quantized large language models*, Nov. 2024. arXiv: 2411.10656 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2411.10656>.
- [12] M. Mozaffari, A. Yazdanbakhsh, and M. M. Dehnavi, *Slim: One-shot quantization and sparsity with low-rank approximation for llm weight compression*, Oct. 2024. arXiv: 2410.09615 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2410.09615>.
- [13] I. Proskurina, L. Brun, G. Metzler, and J. Velcin, *When quantization affects confidence of large language models?* May 2024. arXiv: 2405.00632 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2405.00632>.
- [14] D. Wang and H. Yang, *Taming sensitive weights: Noise perturbation fine-tuning for robust llm quantization*, Dec. 2024. arXiv: 2412.06858 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2412.06858>.
- [15] T.-Y. Chang, M. Zhang, J. Thomason, and R. Jia, *Why do some inputs break low-bit llm quantization?* May 2025. arXiv: 2506.12044 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2506.12044>.
- [16] K. Chen et al., *Q-resafe: Assessing safety risks and quantization-aware safety patching for quantized large language models*, 2025. arXiv: 2506.20251 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2506.20251>.
- [17] B. Chmiel, M. Fishman, R. Banner, and D. Soudry, *Fp4 all the way: Fully quantized training of llms*, May 2025. arXiv: 2505.19115 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2505.19115>.
- [18] P. Czako et al., “Addressing activation outliers in llms: A systematic review of post-training quantization techniques,” *IEEE Journals & Magazine*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10994764>.
- [19] X. Hu et al., *Ostquant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting*, Jan. 2025. arXiv: 2501.13987 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2501.13987>.
- [20] J. Park, T. Lee, C. Yoon, H. Hwang, and J. Kang, *Outlier-safe pre-training for robust 4-bit quantization of large language models*, Jun. 2025. arXiv: 2506.19697 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2506.19697>.
- [21] Q. Wang et al., *Through a compressed lens: Investigating the impact of quantization on llm explainability and interpretability*, May 2025. arXiv: 2505.13963 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2505.13963>.